

ЛЕКЦИЯ 10

JS ФРЕЙМВОРКИ

ВОПРОСЫ

1. JavaScript фреймворки
2. VUE vs REACT
3. React
4. Vue

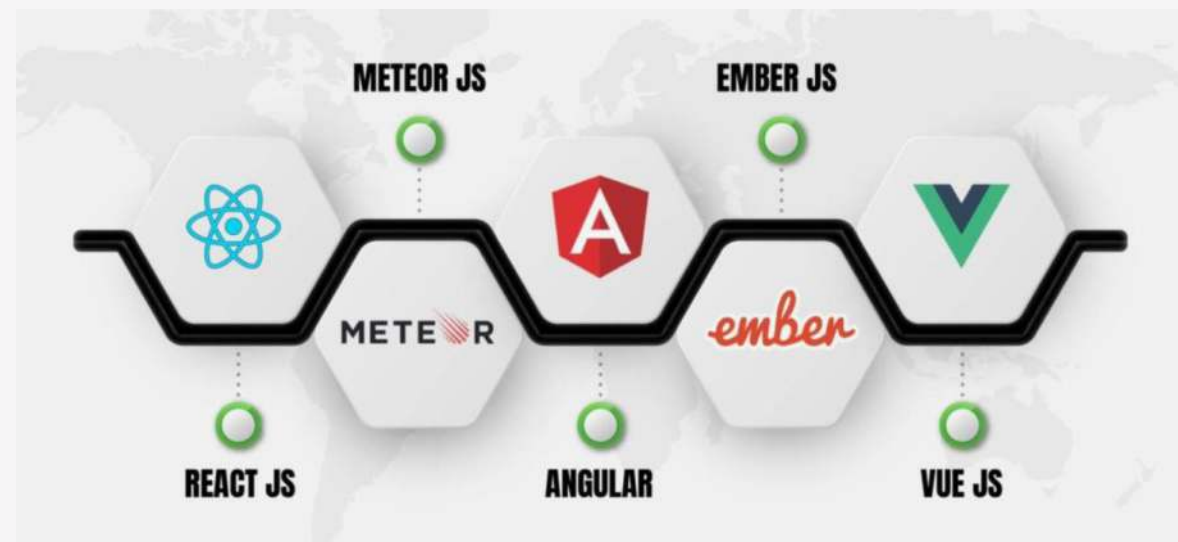
1 JAVASCRIPT ФРЕЙМВОРКИ

JavaScript фреймворки

Понятие JS-фреймворков

Фреймворки JS - это библиотеки программирования JavaScript, в которых есть предварительно написанный код для использования в стандартных функциях и задачах программирования. Это основа для создания веб-сайтов или веб-приложений вокруг.

Давайте начнем с того, зачем нам нужны фреймворки JavaScript? Кодирование вполне возможно без их использования, но правильно подобранная среда может значительно облегчить работу. Более того, они бесплатные и с открытым исходным кодом, так что нет риска.



Зачем нужны JS-фреймворки

В первую очередь это повысит вашу производительность. Рассматривайте это как своего рода обходной путь: вам придется писать меньше кода вручную, потому что уже есть заранее написанные и готовые к использованию функции и шаблоны. Некоторые компоненты веб-сайта не должны быть изготовлены по индивидуальному заказу, поэтому вы можете создавать и расширять предварительно созданные компоненты. Фреймворки более адаптируемы для дизайна веб-сайтов, и большинство разработчиков сайтов предпочитают их.

Frontend Javascript Frameworks





Но как же выбрать JS-фреймворк?

При выборе JS-фреймворка для проекта мы сталкиваемся с необходимостью учитывать целый комплекс факторов. В первую очередь стоит обратить внимание на зрелость экосистемы: количество доступных библиотек, инструментов и готовых решений может существенно повлиять на скорость разработки.

Не менее важным критерием является активность сообщества разработчиков. Фреймворк с большим и активным комьюнити обеспечивает более быстрое решение возникающих проблем, регулярные обновления и богатую документацию.

Производительность и масштабируемость также играют ключевую роль. Например, если проект предполагает обработку больших объемов данных или сложные интерактивные интерфейсы, следует обратить особое внимание на механизмы оптимизации рендеринга и управления состоянием приложения.

Сравнение основных JavaScript фреймворков

Критерий	React	Angular	Vue.js	Svelte
Разработчик	Meta	Google	Evan You	Rich Harris
Год создания	2013	2016	2014	2016
Размер (gzip)	~42.2 KB	~143 KB	~33.2 KB	~1.9 KB
Порог входа	Средний	Высокий	Низкий	Низкий
Virtual DOM	Да	Нет	Да	Нет
Типизация	Flow/TypeScript	TypeScript	TypeScript	TypeScript
Основные use-cases	• Крупные SPA • Динамические приложения • Кросс-платформенная разработка	• Корпоративные приложения • Сложные бизнес-системы • Масштабные проекты	• Небольшие и средние проекты • Прототипирование • PWA	• Легковесные приложения • Высоконагруженные интерфейсы • Быстрые MVP
Особенности производительности	Высокая за счет Virtual DOM	Средняя, компенсируется оптимизацией	Высокая за счет Virtual DOM	Высокая, за счет компиляции в нативный JavaScript

Производительность и масштабируемость

При оценке производительности JS-фреймворков необходимо учитывать несколько ключевых метрик: скорость первоначальной загрузки, время до интерактивности (TTI) и эффективность обновления DOM. Каждый фреймворк предлагает свой подход к оптимизации этих показателей.

Метрика	React	Angular	Vue.js	Svelte
Размер первоначальной загрузки	Средний	Большой	Малый	Минимальный
Время до интерактивности (TTI)	3.2с	4.1с	2.9с	2.3с
Скорость обновления DOM	Быстрая	Средняя	Быстрая	Очень быстрая
Потребление памяти	Среднее	Высокое	Низкое	Минимальное
Масштабируемость	Отличная	Отличная	Хорошая	Хорошая

2

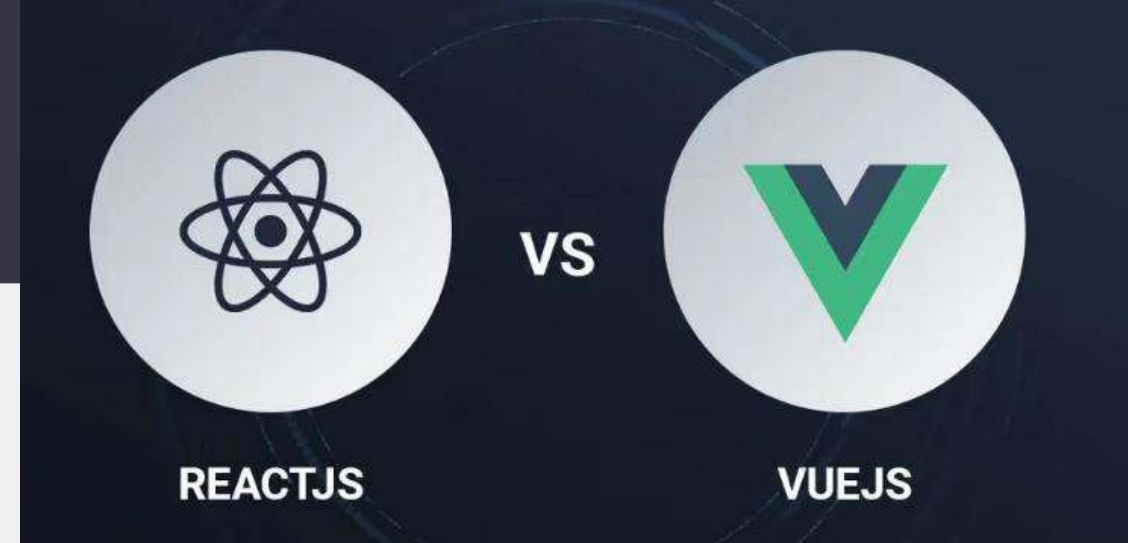
VUE VS REACT

VUE VS REACT

JavaScript все чаще признается основным скриптовым языком для разработки веб-приложений. Благодаря адаптивности и соответствию веб-стандартам он пользуется популярностью среди разработчиков. Фреймворки на JavaScript, предлагающие готовые структуры и функциональные возможности, упрощают процесс разработки. Они берут на себя рутинные задачи, позволяя разработчикам сосредоточиться на уникальных элементах своих приложений. Среди таких фреймворков часто сравнивают Vue и React, которые являются популярными поисковыми запросами во всем мире.

Существует множество JS-фреймворков, каждый из которых имеет свои преимущества и ограничения. Выбор фреймворка в первую очередь зависит от конкретных потребностей проекта. В сообществе разработчиков часто спорят о том, какой фреймворк лучше - Vue или React, и какой из них выбрать?

Прежде чем изучать какой-либо фреймворк, необходимо разобраться в фундаментальных концепциях JS. Эти базовые знания важны для эффективного использования и освоения JS-фреймворка, особенно Vue и React.



VUE VS REACT. Кратко

И ReactJS, и VueJS - мощные инструменты для создания интерактивных пользовательских интерфейсов, каждый из которых имеет свои уникальные преимущества.

При выборе между Vue и React следует исходить из четкого понимания потребностей вашего проекта, возможностей команды и долгосрочных целей.

Оба инструмента можно использовать практически везде, где применяется Vue или React, но иногда один из них проще в использовании, чем другой.

В таблице ниже приведены факторы, которые следует учитывать при выборе между React и Vue.

Факторы, которые следует учитывать	ReactJS	VueJS
Гибкость	ДА	ДА
Производительность	НЕТ	ДА
Большие сообщества	ДА	НЕТ
Низкая кривая обучения	НЕТ	ДА
Световой каркас	НЕТ	ДА
Для создания сложных приложений	ДА	НЕТ
Динамические приложения	ДА	ДА
Чистые HTML-файлы	НЕТ	ДА

1 VUE VS REACT. Порог входа

React

Высокий: требуется знание JSX и современной JS-парадигмы. Новичкам сложнее начать (хуки, экосистема).

Vue

Низкий: классический шаблонный синтаксис, понятный веб-разработчикам. Старт прост – базовые вещи работают «из коробки».



2 VUE VS REACT. Документация и обучение

React

Полноценная обновленная документация, но охватывает только ядро; за остальным нужно обращаться к сторонним руководствам.

Много учебных материалов, курсов.

Vue

Отличная официальная документация, детально покрывающая все разделы (Vue 3 docs высоко оцениваются разработчиками).

Чёткая структура облегчает самообучение.



3 VUE VS REACT. Гибкость архитектуры

React

Максимальная гибкость:

React - только UI-слой.

Можно подключить любые

библиотеки (роутер, state, анимации) по своему вкусу.

Это даёт свободу, но требует больше решений и договорённостей в команде.

Vue

«Конвенция над конфигурацией»:

Vue сразу предоставляет официальные решения (роутер, хранилище).

Меньше необходимости выбирать инструменты - инфраструктура более унифицирована. Чуть меньше свободы, но выше согласованность.



4 VUE VS REACT. Синтаксис и компоненты

React

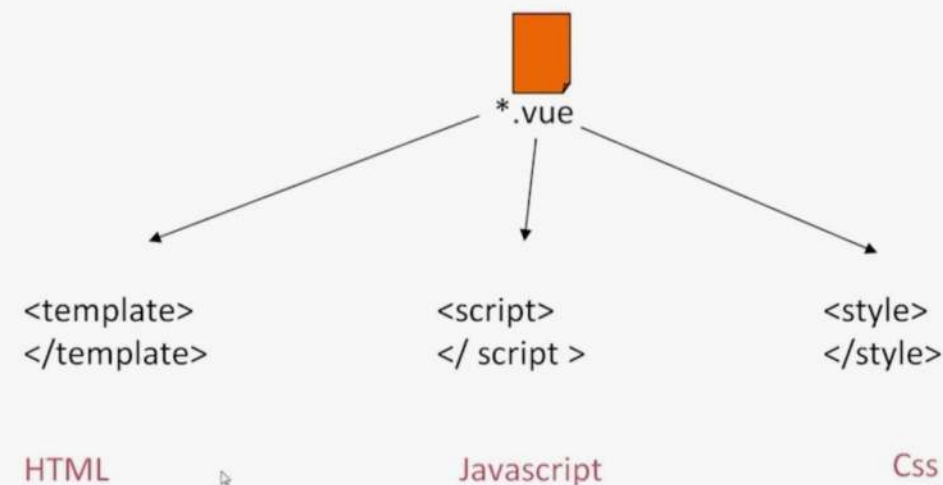
JSX + JS: логика и разметка объединены. Очень мощно (условия, тар и любой JS прямо в шаблоне), но непривычно для начинающих.

Компоненты - классовые (в прошлом) или функциональные с хуками. Требуется транпиляция JSX (Babel).

Vue

SFC (Single File Component): разделение на `<template>``<script>``<style>`. Шаблоны похожи на HTML, с директивами (v-if, v-for). Меньше «магии» JS, зато код более декларативен.

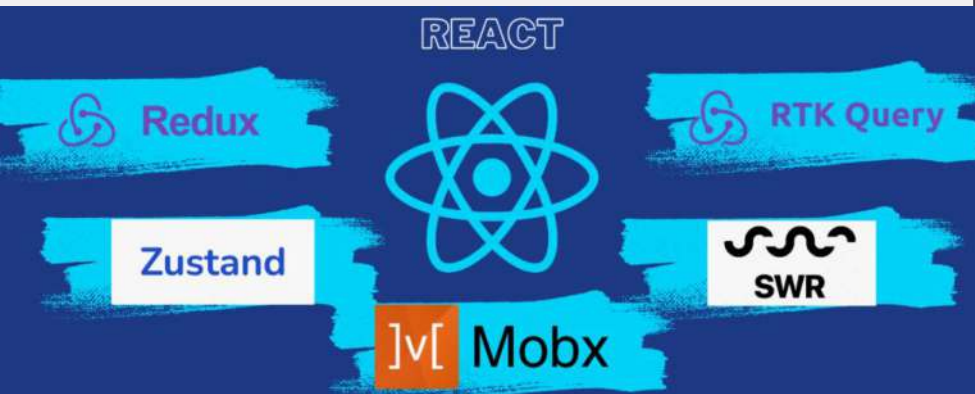
Можно писать также в режиме JSX/TSX при желании, но основной подход – шаблоны.



5 VUE VS REACT. Управление состоянием

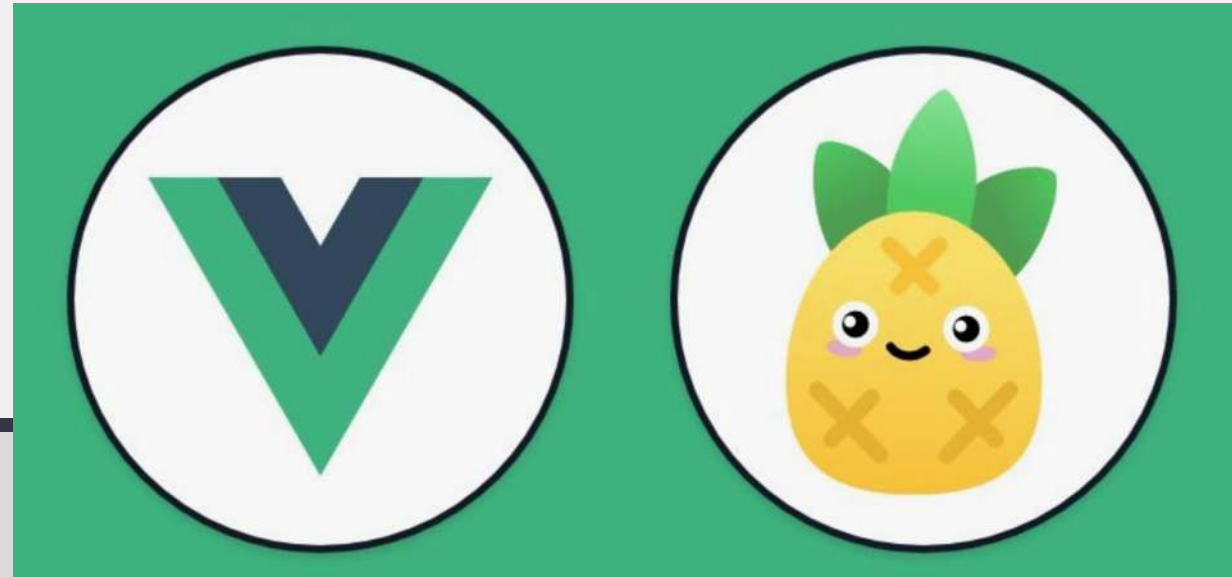
React

Нет встроенного глобального состояния. На выбор Redux, MobX, Context API или современные легковесные библиотеки (Zustand, Jotai). Много вариантов под разные задачи. В локальном состоянии - хуки (useState, useReducer).



Vue

Официальное глобальное хранилище: Pinia (замена Vuex) - интегрировано с Vue 3. Простое API, TypeScript-совместимость, реактивность. Меньше шаблонного кода, чем Redux. Подходит для 90% случаев. Локальное состояние - реактивные свойства через Composition API (ref, reactive).



6 VUE VS REACT. Двухнаправленность данных

React

Однонаправленный data flow:
изменяем state ->
перерисовываются компоненты.
Двухнаправленных биндингов нет
(форма -> state через onChange).

Это делает данные
предсказуемыми, хотя требует
писать доп. код.

Vue

Поддерживается двустороннее связывание через v-model
для удобства форм. Компоненты могут эмитить событие
update:modelValue. Ускоряет разработку UI, но чрезмерное
использование может запутать поток данных, поэтому
применяется умеренно.

7 VUE VS REACT. Performance (рендеринг)

React

Высокая производительность благодаря Virtual DOM и оптимизации diff. React 18/19 добавил Concurrent Rendering для плавности больших обновлений. Требуется ручная оптимизация в критичных местах (мемоизация). Подходит для очень сложных интерфейсов без существенных проблем.

Vue

Также очень быстрый: Virtual DOM + реактивность. Vue 3 оптимизировал скорость по сравнению с Vue 2, уменьшил размер (ядро ~34KB gzip). Реактивная система обновляет только необходимые компоненты. В небольших приложениях практически мгновенный рендер. Обе технологии сопоставимы по скорости.

8 VUE VS REACT. Масштабируемость проекта

React

Хорошо подходит для больших проектов с десятками разработчиков. Гибкая архитектура React и богатый экосистемой инструмент (например, Redux) позволяют масштабировать приложение без потери структуры. Используется во многих enterprise-приложениях.

Vue

Подходит для малых и средних приложений изначально. Крупные проекты тоже возможны (есть примеры), однако чаще Vue выбирают для менее сложных систем.

На больших проектах нужна дисциплина (модульность, разделение на под-приложения). В целом Vue 3 в сочетании с TypeScript и Pinia стал более пригоден для масштабирования, чем Vue 2.

9 VUE VS REACT. SSR/SSG (серверный рендер)

React

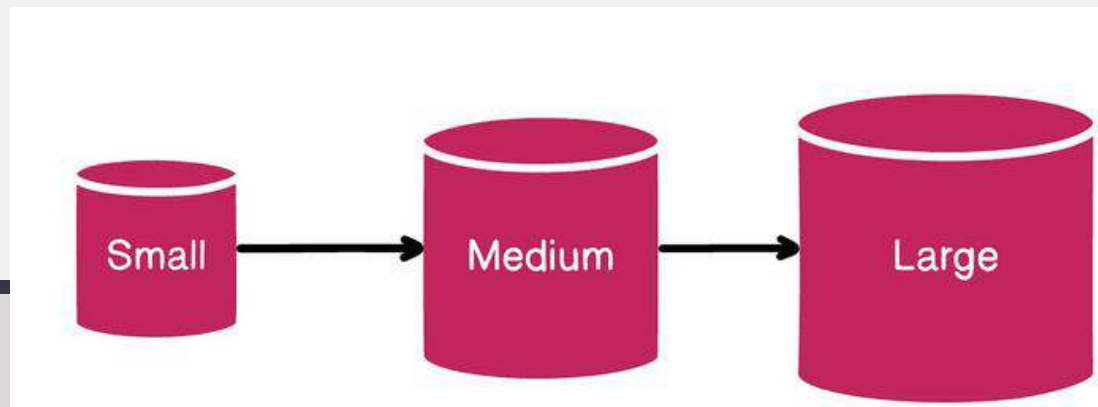
Благодаря Next.js - богатые возможности: SSR, SSG, ISR (инкрементальная генерация). React+Next позволяет строить гибридные приложения (часть страниц SSR, часть статически).

Есть и альтернативы: Gatsby для SSG, Remix для SSR. Next.js очень зрелый инструмент с множеством функций (образец – встроенные API маршруты и оптимизация изображений).

Vue

Через Nuxt.js – аналогичный функционал: SSR, SSG (режим static). Nuxt 3 догнал Next по большинству возможностей, включая SSR/SSG на уровне страниц, prefetching, middleware. В Nuxt меньше гибкости (следует конвенциям), но быстрый старт.

Без Nuxt чистый Vue тоже может SSR (framework-agnostic SSR), но практически все выбирают Nuxt для этих целей.



10 VUE VS REACT. Интеграция с backend

React

Богатый выбор подходов: REST, GraphQL, RPC – любая схема через fetch/axios. В Next можно писать backend-логику прямо в проекте (API Routes), что упрощает интеграцию (например, обрабатывать формы на сервере). Множество библиотек для работы с данными: React Query, SWR и т.д., повышающих эффективность работы с API.

Vue

Аналогично: Vue сам по себе агностичен к способу API.

Через Nuxt 3 можно создавать server endpoints (Nitro) – тем самым интегрировать бэкенд-логику. Для Vue доступны Apollo (GraphQL) и другие клиенты. Реактивность Vue облегчает синхронизацию с real-time: просто обновляем объект – UI меняется. Есть обертки вроде vue-query для удобной работы с серверными данными.

11 VUE VS REACT. TypeScript поддержка

React

Отлично поддерживается: имеется `@types/react`, многие библиотеки `typed`. React 18 улучшил типизацию Hooks (например, `useReducer` с `ActionTypes`). Нет собственного компилятора – просто использует `tsc/babel`. Компоненты можно писать на TSX.

Vue

Vue 3 изначально написан на TS, что улучшило типы. С помощью Volar IDE подсказывает типы даже внутри шаблонов.

Pinia и остальные официальные пакеты имеют полный TypeScript support. Режим `class-based API` (через `decorators`) устарел, сейчас рекомендуют Composition API + TS, что дает сильную типизацию.

12 VUE VS REACT. Инструменты сборки

React

Раньше CRA (Webpack) - удобство старта, но медленная переработка при развитии. Сейчас он официально снят с рекомендации react.dev, вместо него - Next.js или Vite.

С Vite запуск React-проекта практически мгновенный, HMR быстрый. Webpack все еще исп-ся внутри Next, но абстрагирован. Экосистема смещается на Vite, Parcel, Turbopack (новый сборщик от Vercel).

Vue

Всегда имел CLI на Webpack, но Vue 3 мгновенно адаптировался к Vite. create-vue по умолчанию генерирует Vite-конфигурацию. Это ускорило разработку Vue-проектов (быстрый старт сервера, меньше проблем с конфигом). Интеграция с build-tool на высоте: например, Vue CLI UI (в Vue 2) позволял графически управлять настройками. Сейчас Vite-плагины заменяют необходимость глубокой ручной настройки.



Vite

Next Generation Frontend Tooling

13 VUE VS REACT. DevTools и отладка

React

React DevTools (расширение) - показывает дерево компонентов, состояния hook'ов, пропсы.

React Profiler - для анализа произв-ти.

ESLint плагины - контролируют правила хуков и прочее. Интеграция с Chrome хорошая. Большое комьюнити значит, что ответы на любую проблему легко найти (StackOverflow, блоги и т.п.).

Vue

Vue DevTools – удобное расширение, показывает компонентное дерево, реактивные данные, Vuex/Pinia стор с time-travel (в режиме разработки). Поддерживает Vue 3 Composition API. Также есть Vue.js devtools для Firefox/Chrome.

Сообщество Vue достаточно дружелюбно: вопросы решаются в форумах, официальном чате. Документация часто покрывает и отладочные моменты.

14 VUE VS REACT. Тестирование

React

React Testing Library - популярна для тестирования компонентов через виртуальный DOM (позволяет проверять, что и как рендерится, реагирует на события). Jest - для юнит-тестов логики. Cypress или Playwright - для e2e тестов (подходит и для React, и для Vue одинаково). В целом, тестовый экосистемы React более зрелая (RTL стала стандартом, много статей).

Vue

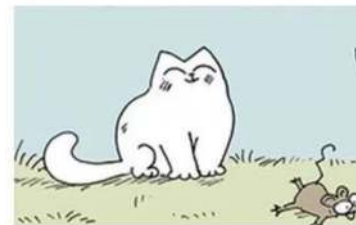
Для Vue - `@vue/test-utils` предоставляет методы рендеринга компонентов в изоляции, эмита событий и проверки результатов. Используется вместе с Jest. Подход похож на RTL, но специфичен для Vue.

Также поддерживаются снапшоты, shallow-монтирование. Cypress имеет плагин для компонентного тестирования Vue. Разница несущественна: и React, и Vue приложения можно тестировать надежно.

Начинающий тестировщик и баг



Опытный тестировщик и баг



15 VUE VS REACT. Мобильная разработка

React

Огромный плюс React - React Native. Это фреймворк для кроссплатформенной разработки, широко применяемый (Instagram, Shopify на RN). Знание React позволяет писать мобильные приложения с нативным UI. Кроме того, есть Electron и React Native Web – расширяя возможности React в десктоп и PWA.

Vue

У Vue тоже есть варианты, но менее развитые.

NativeScript-Vue – интеграция Vue с NativeScript для мобильных приложений, однако сообщество маленькое, и решение не получило такого распространения, как RN. Также существуют Capacitor и Cordova для Vue (веб-вью). Но в сфере мобильного разработки React явно лидирует как более зрелый путь.



16 VUE VS REACT. Экосистема UI-библиотек

React

Неисчислимо кол-во библиотек компонентов и дизайна: Material-UI (MUI), Ant Design, Chakra UI, Bootstrap React, Semantic UI – под любую задачу. Анимации: Framer Motion, React Spring. Формы: Formik, React Hook Form. Практически любая потребность уже закрыта готовым React-пакетом. Это ускоряет разработку, но требует совместимости версий и может приводить к доп. зависимостям.

Vue

У Vue экосистема компонентов тоже выросла. Популярны фреймворки: Vuetify (Material Design), Element Plus, Ant Design Vue, Quasar (целый фреймворк для SPA/PWA), BootstrapVue. Анимации: Vueuse Motion. Формы: Vuelidate, Vue Formulate. Выбор меньше, чем у React, но ключевые потребности покрыты. Большинство популярных библиотек обновилась под Vue 3. Новые библиотеки часто более легковесны и интегрированы с Composition API.

17 VUE VS REACT. Сообщество и поддержка

React

Огромное глобальное сообщество. Быстрые ответы на вопросы, множество блогов, видео-каналов (хотя информация бывает избыточна или устаревает - например, много старых туториалов с классовыми компонентами).

Поддерживается крупными корпорациями - продолжает развиваться активно (React 19 выпущен в 2024).

Vue

Сообщество меньше, но очень активное и сплоченное. Vue популярен в азиатских странах, есть локальные сообщества.

Создатель Эван Ю напрямую общается с комьюнити (через RFC, Twitter).

Развитие поддерживают компании (например, Sponsor: Netlify, Ionic). Обновления регулярны и учитывают фидбэк (Vue 3.3, 3.4 в 2023 добавили удобства, запрошенные разработчиками).

Лучи поддержки



18 VUE VS REACT. Enterprise-применение

React

Частый выбор для Enterprise. React используется в продуктах Facebook, Netflix, Uber, Airbnb и тысяч других. Большие компании ценят его гибкость и широкую поддержку. Существуют крупные консалтинг-фирмы специализации React, много инженеров на рынке. Вакансий по React больше, чем по любому другому фронтенд-фреймворку.

Vue

Хотя Vue чаще ассоциируется со стартапами, его применяют и большие игроки: Alibaba, Xiaomi, Tencent в Китае; в Европе – BMW, Samsung в отдельных приложениях, Nintendo (частично), GitLab. Enterprise-внедрение растет по мере того, как Vue 3 доказал стабильность. Однако в корпоративном сегменте React и Angular пока опережают Vue по доле.

19 VUE VS REACT. Обновления и изменения

React

Очень стабильный API. С 16-й версии до 18-й не было крупных ломающих изменений – код обновляется плавно. Новые возможности (Hooks в 16.8) добавляются с сохранением обратной совместимости. Даже React 19 ориентирован на постепенные улучшения, а не революцию. Это снижает риск при выборе React – инвестированный код устаревает медленно.

Vue

Переход от Vue 2 к Vue 3 - значительное изменение (новый механизм реактивности, Composition API). Хотя команда обеспечила постепенную миграцию (долгое сосуществование Vue 2 и 3, адаптеры), разработчикам пришлось обновлять код и библиотеки. Теперь, после завершения миграции (2022), Vue обещает более плавные эволюции. Т.е. в ближайшие годы резких ломающих изменений не планируется – только дополняющие.

20 VUE VS REACT. Философия и подход

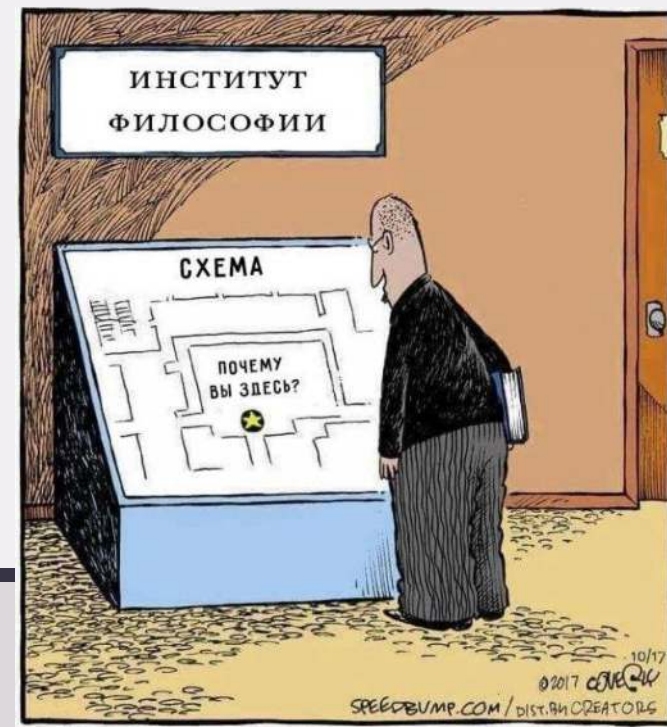
React

Либеральный подход - библиотека, собираемая под потребности проекта. React менее мнителен к архитектуре - можно писать по-разному.

Основная философия - $UI = f(state)$, UI как функция от состояния, и управление состоянием выведено наружу. Отличается тем, что смешивает разметку с логикой (JSX), что можно трактовать как отход от классического MVC, на вкус команды.

Vue

Прогрессивный подход - начать с малого, расширять по необходимости. Можно вклинивать Vue постепенно в существующую страницу. Разделение ответственностей сохраняется: шаблон + код + стили. Философия Vue - сделать простые вещи простыми, сложные - возможными, что достигается балансом между встроенным функц-м и возм-ю подключать плагины. Vue также акцентирует удобство разработчика (синтаксический сахар вроде v-model или `<script setup>` для сокращения кода).



21 VUE VS REACT. Итого позиционирование

React

- это выбор для крупных и долгосрочных проектов, где важны производительность, гибкость и огромная экосистема. Имеет поддержку корпораций, богатый рынок труда, множество готовых решений. Иногда требует больше опыта, зато практически нет задач, которые нельзя решить средствами React-экосистемы.

Vue

- это выбор для быстрых результатов, стартапов, или проектов, где команда ценит простоту и интегрированные инструменты. Позволяет с минимальными усилиями создать полноценное приложение. Отлично подходит для SPA, административных панелей, где нужен быстрый отклик. В больших системах тоже работает, особенно с приходом Vue 3, но это относительно новая практика для многих компаний.

3

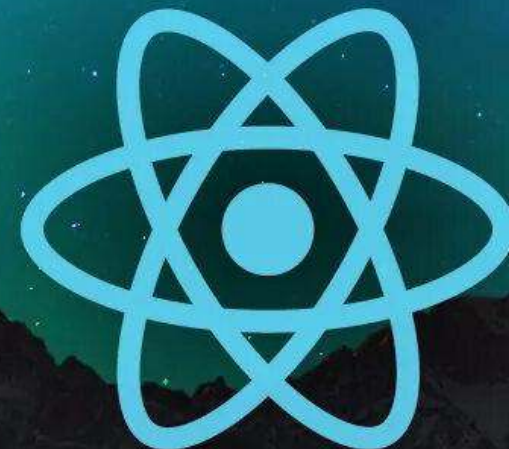
REACT

Введение в React и окружение

React – это библиотека для построения пользовательских интерфейсов, которая позволяет создавать сложные веб-приложения с минимальными усилиями по ручному управлению DOM.

Прежде чем начать работу, необходимо подготовить окружение: установить Node.js, который обеспечивает запуск инструментов сборки и дев-сервера, а также менеджер пакетов npm, идущий в комплекте с Node.js.

Для разработки рекомендуется использовать редактор VS Code с расширениями ESLint, Prettier и React Developer Tools для браузера.



Введение в React и окружение

Создание нового проекта выполняется с помощью Vite – инструмента сборки, который генерирует готовую структуру папок и файлов. Команда ***npm create vite@latest my-app --template react-swc*** создает проект с именем my-app, используя шаблон React с быстрым компилятором SWC. После создания нужно перейти в папку проекта, установить зависимости через `npm install` и запустить сервер разработки командой `npm run dev`. Для сборки готового приложения используется `npm run build`, а проверить собранную версию локально можно через `npm run preview`.

```
PS D:\GFG> npm create vite@latest my-react-app --template react
? Select a framework: » - Use arrow-keys. Return to submit.
  Vanilla
  Vue
>  React
  Preact
  Lit
  Svelte
  Solid
  Qwik
  Others
```

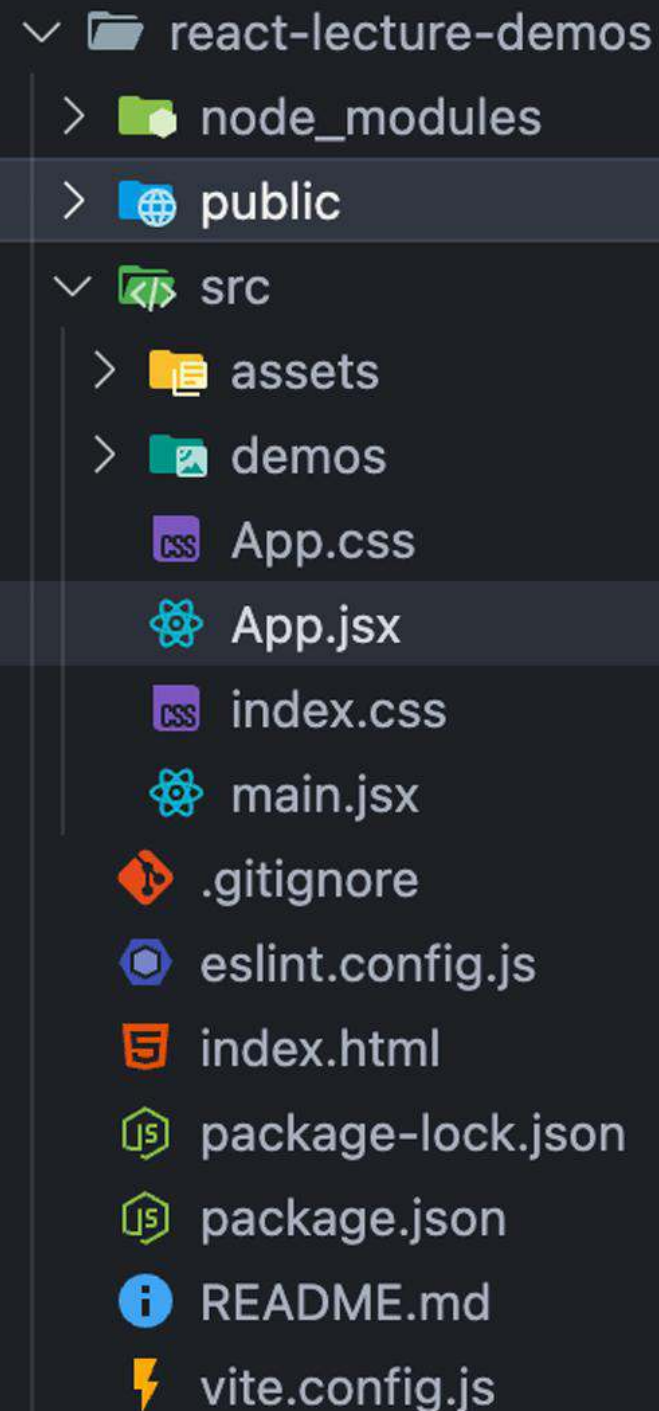
React

Введение в React и окружение

```
my-app/  
├─ node_modules/  
├─ public/  
├─ src/  
│   ├─ main.jsx  
│   ├─ App.jsx  
│   ├─ App.css  
│   └─ index.css  
├─ index.html  
├─ package.json  
├─ vite.config.js  
└─ .gitignore
```

Структура проекта организована логично: в папке `src` находится исходный код, `main.jsx` служит точкой входа, `App.jsx` представляет корневой компонент, а `index.html` содержит контейнер с идентификатором `root`, куда React монтирует приложение.

Важно понимать, что в современной React-разработке мы используем не Create React App, а Vite, поскольку он предлагает значительно более быструю работу и простую конфигурацию.



React создан для построения одностраничных приложений, где в отличие от классических многостраничных сайтов с отдельными HTML-файлами, загружается одна страница, а вся дальнейшая навигация происходит без перезагрузки за счет подгрузки данных через API и динамического обновления интерфейса.

Это дает ощущение плавного, отзывчивого приложения. Яркими примерами таких приложений являются почтовые сервисы и личные кабинеты банков.



SPA

React

В основе React лежит декларативная парадигма – вместо того чтобы писать императивные инструкции по поиску элементов DOM и их изменению, мы просто описываем, как интерфейс должен выглядеть при текущих данных. Формула UI равна функции от состояния и пропсов означает, что интерфейс полностью определяется входными данными, а React сам заботится о том, чтобы привести DOM к нужному виду.

Это упрощает разработку и уменьшает ошибки.



Декларативный подход

React

JSX представляет собой синтаксическое расширение JavaScript, позволяющее писать HTML-подобную разметку прямо в коде компонентов. При этом важно помнить несколько правил: компонент всегда должен возвращать один корневой элемент, для чего можно использовать фрагменты в виде пустых угловых скобок.

Внутри фигурных скобок можно встраивать любые JavaScript-выражения – переменные, вычисления, тернарные операторы и вызовы функций, но нельзя использовать инструкции вроде if или for.

JSX имеет отличия от обычного HTML: вместо class используется className, вместо for – htmlFor, имена событий пишутся в camelCase. Стили задаются объектом, где значениями выступают строки или числа.

JSX

React

Simpler render functions

JSX

```
render(h) {  
  return (  
    <BaseInput  
      vModel={this.searchText}  
      class={this.$style.searchInput}  
      placeholder="Search"  
    />  
  )  
}
```

without JSX

```
render(h) {  
  return h(BaseInput, {  
    props: {  
      value: this.searchText  
    },  
    on: {  
      update: newValue => {  
        this.searchText = newValue  
      }  
    },  
    class: this.$style.searchInput,  
    attrs: {  
      placeholder: 'Search'  
    }  
  })  
}
```



JSX with React 🤗

Index.html

```
1 <html>
2   <head>
3     <meta charset="utf-8" />
4     <title>잘라먹는 React.js - About</title>
5   </head>
6   <body>
7     <!-- HEADER -->
8     <MyHeader />
9
10    <!-- NAV -->
11    <MyNav />
12
13    <!-- ARTICLE -->
14    <article>
15      <h3 style="color: red;">여기는 ABOUT 입니다</h3>
16    </article>
17
18    <!-- FOOTER -->
19    <footer />
20  </body>
21 </html>
```

MyHeader 컴포넌트

```
<!-- HEADER -->
<header>
  <h1>안녕하세요</h1>
  <b>저는 유데미에서 React.js를 강의하는 winterlood 이정환입니다</b>
</header>
```

MyNav 컴포넌트

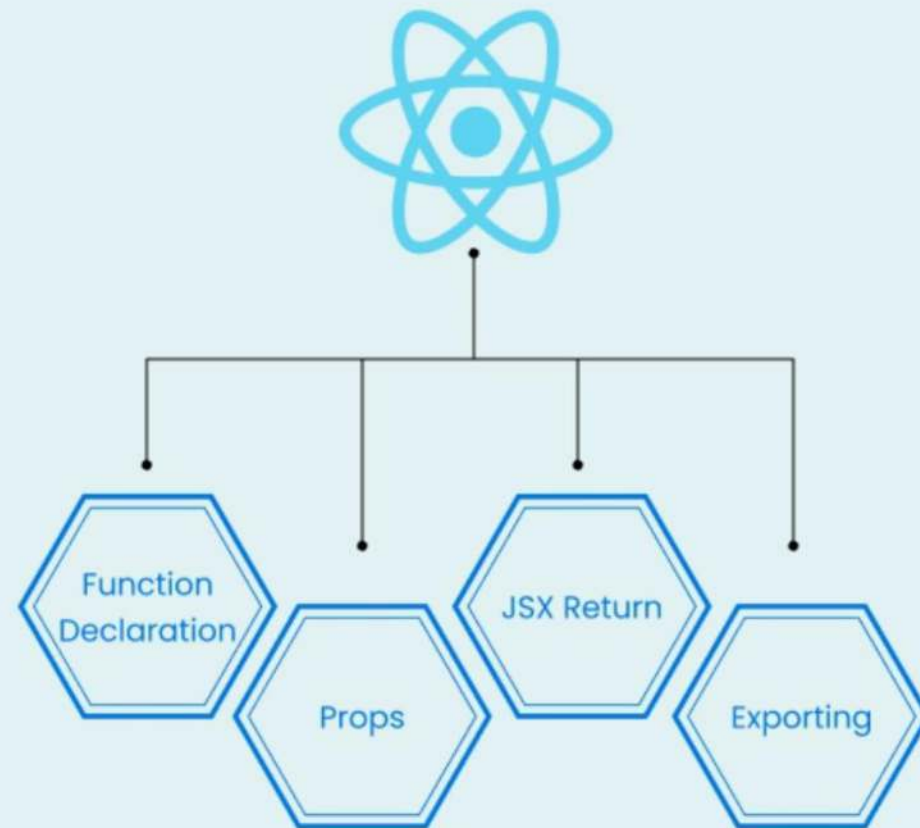
```
<!-- NAV -->
<nav>
  <ul>
    <a href="/index.html"><li>Home</li></a>
    <a href="/about.html"><li>About</li></a>
  </ul>
</nav>
```

MyFooter 컴포넌트

```
<!-- FOOTER -->
<footer>
  <p>방문해주셔서 감사합니다</p>
</footer>
```

Компоненты

```
1 import React from "react";
2 import "./App.css";
3
4 function App() {
5   return (
6     <div className="App">
7       <header className="App-header">
8         <h2>Hello From React</h2>
9       </header>
10     </div>
11   );
12 }
13
14 export default App;
```



Функциональный компонент в React – это чистая функция, которая принимает объект пропсов и возвращает JSX. Чистота означает, что при одинаковых входных данных компонент всегда возвращает одинаковый результат и не производит побочных эффектов – не изменяет внешние переменные, не делает сетевых запросов, не запускает таймеры. Это делает компоненты предсказуемыми, легко тестируемыми и безопасными для оптимизаций React.

React

Пропсы

```
class PetsList extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = {  
      pets: pets  
    };  
  }  
  render () {  
    return (  
      <div>  
        <h3>Pets List: </h3>  
        {  
          this.state.pets.map((v, idx) => {  
            return <SinglePet key={idx}  
              name={v.name}  
              image={v.image}  
              desc={v.desc} />  
          })  
        }  
      </div>  
    )  
  }  
}
```

```
class SinglePet extends React.Component {  
  render () {  
    return (  
      <div>  
        <h3>{this.props.name}</h3>  
        <img src={this.props.image} alt="" />  
        <p>{this.props.desc}</p>  
      </div>  
    )  
  }  
}
```

state

props

Пропсы (входные параметры), передаются от родительского компонента дочернему и являются read-only – дочерний компонент никогда не должен их изменять. Для удобства чтения кода принято использовать деструктуризацию пропсов прямо в параметрах функции.

Особый пропс children позволяет реализовать композицию компонентов, вкладывая один компонент в другой подобно тому, как мы вкладываем HTML-теги. Это мощный механизм для создания гибких и переиспользуемых компонентов вроде карточек, панелей или кнопок с разными вариантами оформления.

Условный рендеринг в React реализуется несколькими способами в зависимости от ситуации. Для опционального показа элемента используется логическое И с оператором `&&`. Для выбора между двумя вариантами применяется тернарный оператор. Для полного скрывтия компонента можно вернуть `null`. При работе с данными важно обрабатывать состояния загрузки, ошибок и пустого списка – это делается через последовательные проверки с возвратом соответствующих компонентов на каждом этапе.

```
if (condition) {  
  return <p>Expression 1</p>;  
} else {  
  return <p>Expression 2</p>;  
}
```

```
return (  
  <div>  
    {shouldRenderComponentA ? <ComponentA /> : <ComponentB />}  
  </div>  
);
```

Опциональные куски: `isVisible && <Block />`

Выбор из двух: `cond ? <A /> : <B />`

Дефолт только для `null/undefined`: `value ?? fallback`

Полное скрывтие: `if (!show) return null;`

Фрагменты без лишнего контейнера: `<>...</>`

Условный рендеринг

При отрисовке списков через метод `map` критически важно указывать атрибут `key` на корневом элементе каждого элемента. `Key` помогает React идентифицировать элементы при изменениях списка - добавлении, удалении или перестановке. В качестве `key` нужно использовать стабильный уникальный идентификатор из данных, например идентификатор элемента. Использование индекса массива в качестве `key` допустимо только для статических списков, которые никогда не меняются, поскольку при перестановках это приводит к ненужным перерисовкам и потере локального состояния.

```
return (  
  <>  
    {errorMessage || isLoading ? (  
      <p>{errorMessage || 'Loading...'}</p>  
    ) : (  
      <ul>  
        {data.map((item) => (  
          <li key={item.id}>{item.name}</li>  
        ))}  
      </ul>  
    )}  
  </>  
);
```

```
return (  
  <Router>  
    <Switch>  
      <Route exact path="/" component={Home} />  
      <Route path="/login">  
        <Login setIsLoggedIn={setIsLoggedIn} />  
      </Route>  
      {isLoggedIn ? (  
        <Route path="/dashboard" component={Dashboard} />  
      ) : (  
        <Route component={NotFound} />  
      )}  
    </Switch>  
  </Router>  
);
```

Условный рендеринг и списки

События

React использует собственную обертку над нативными событиями – `SyntheticEvent`, которая обеспечивает одинаковое поведение во всех браузерах. События проходят две фазы: захват, идущий сверху вниз, и всплытие, идущее снизу вверх. По умолчанию обработчики срабатывают на фазе всплытия, но можно подписаться и на фазу захвата, используя суффикс `Capture`. В обработчике доступны свойства `target` – элемент, на котором произошло событие, и `currentTarget` – элемент, на котором сработал обработчик.



События

Для управления поведением событий используются методы `preventDefault`, отменяющий действие по умолчанию например перезагрузку страницы при отправке формы, и `stopPropagation`, останавливающий дальнейшее всплытие события.

В формах важно правильно указывать тип кнопок: кнопка без указанного типа или с `type="submit"` вызывает отправку формы, кнопка с `type="button"` предназначена для второстепенных действий, а `type="reset"` сбрасывает все поля.

```
buttonElement.addEventListener('click', (event) => {  
  event.stopPropagation(); // Это остановит распространение события.  
});
```

```
formElement.addEventListener('submit', (event) => {  
  event.preventDefault(); // Это предотвратит отправку формы.  
});
```

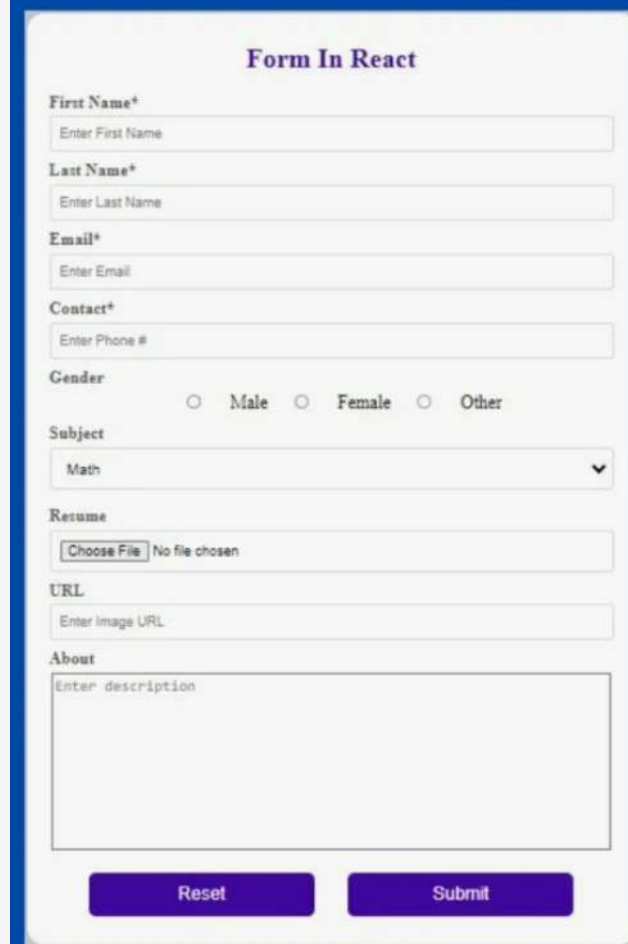
События

Для работы с формами React предлагает два подхода.

Первый – использование FormData, когда данные собираются непосредственно из формы при отправке без хранения их в состоянии. Это простой и эффективный способ для несложных форм.

Второй подход – контролируемые компоненты, где значение каждого поля хранится в состоянии React и синхронизируется через атрибуты value и onChange, что дает полный контроль над вводом и возможность мгновенной валидации.

Для группировки связанных элементов используются теги fieldset и legend, обеспечивающие семантическую и доступную разметку.



The image shows a web form titled "Form In React" with a blue border. The form contains the following fields and controls:

- First Name***: A text input field with placeholder text "Enter First Name".
- Last Name***: A text input field with placeholder text "Enter Last Name".
- Email***: A text input field with placeholder text "Enter Email".
- Contact***: A text input field with placeholder text "Enter Phone #".
- Gender**: Three radio buttons labeled "Male", "Female", and "Other".
- Subject**: A dropdown menu with "Math" selected and a downward arrow.
- Resume**: A file upload control with a "Choose File" button and the text "No file chosen".
- URL**: A text input field with placeholder text "Enter image URL".
- About**: A large text area with placeholder text "Enter description".
- Buttons**: Two purple buttons at the bottom labeled "Reset" and "Submit".

Хук `useState` предоставляет компоненту возможность хранить и обновлять локальное состояние. Он возвращает массив из двух элементов: текущего значения и функции для его обновления.

Состояние нужно использовать только тогда, когда компонент должен запоминать данные между рендерами и изменения этих данных должны влиять на интерфейс. Если значение можно вычислить из пропсов или другого состояния, отдельное состояние не требуется.

Примеры:

- Счётчик кликов
- Открыта ли модалка
- Текст в поле ввода
- Выбранная вкладка
- Список задач

Состояние (state) в React — это данные, от которых зависит вывод компонента.

Изменение состояния $\Rightarrow$ повторный рендер компонента.

Состояние и `useState`

React

```
const App = () => {  
  const [value, valueChange] = useState(0);  
  
  return (  
    <div>  
      {value}  
      <button onClick={() => valueChange(value + 1)}>  
        Увеличить значение на 1  
      </button>  
    </div>  
  );  
};
```

При работе с объектами и массивами в состоянии необходимо соблюдать принцип иммутабельности – никогда не изменять существующие структуры напрямую, а создавать новые копии с помощью spread-оператора, map или filter. Для вложенных объектов требуется копировать каждый уровень до изменяемого свойства. Это необходимо, потому что React определяет изменения по ссылкам, и мутация исходного объекта не вызовет перерендер.

КОГДА ИСПОЛЬЗОВАТЬ useState:

НУЖНО СОСТОЯНИЕ, если:

1. Компонент должен "помнить" данные между рендерами
2. Изменение должно вызвать ререндер (обновить UI)
3. Нельзя вычислить из props или других данных
4. Данные меняются в результате действий пользователя

НЕ НУЖНО СОСТОЯНИЕ, если:

1. Можно вычислить из props:

```
const fullName = `${firstName} ${lastName}`;
```

2. Можно вычислить из другого state:

```
const isValid = password.length >= 8;
```

3. Данные не влияют на UI:

Используйте useRef!

4. Константы и статичные данные:

```
const ITEMS = ['A', 'B', 'C'];
```

Состояние и useState

React

Эффекты и useEffect

```
function GoodComponent() {  
  const [count, setCount] = useState(0);  
  useEffect(() => {  
    document.title = `Счёт: ${count}`;  
  }, [count]);  
  return <div>{count}</div>;  
}
```

Хук `useEffect` предназначен для синхронизации компонента с внешним миром - выполнения побочных эффектов, которые невозможно или неправильно выполнять во время рендера. К таким эффектам относятся подписки на события, таймеры, работа с `localStorage`, сетевые запросы и интеграция с внешними библиотеками. Важно понимать, что эффекты выполняются после того, как React обновит DOM и браузер отрисует изменения.

Каждый эффект может возвращать функцию очистки, которая вызывается перед повторным запуском эффекта или при размонтировании компонента. Это критически важно для предотвращения утечек памяти - таймеры должны очищаться, подписки отменяться, а сетевые запросы прерываться через `AbortController`.

Эффекты и useEffect

Второй аргумент `useEffect` – массив зависимостей – определяет, когда эффект должен перезапускаться. Пустой массив означает выполнение только при монтировании и очистку при размонтировании. Массив с зависимостями вызывает эффект при изменении любой из них. Отсутствие массива приводит к выполнению эффекта после каждого рендера, что редко бывает нужно. Все значения, используемые внутри эффекта, должны быть указаны в зависимостях – это контролируется линтером и помогает избежать трудноотловимых багов.

Эффекты и useEffect

Режим строгой разработки StrictMode в development-среде дважды монтирует компонент с очисткой между монтированиями, чтобы проверить корректность реализации эффектов. Это помогает обнаружить проблемы с очисткой и подготовить код к будущим возможностям React, поэтому игнорировать двойные вызовы не следует.

В DEV со StrictMode:

mount → setup → cleanup → setup

↑ _____ ↑

ДВОЙНОЙ ВЫЗОВ!

В production:

mount → setup (один раз)

Типичные проблемы:

ПРОБЛЕМА: Эффект без cleanup

```
useEffect(() => {  
  const id = setInterval(() => setCount(c => c + 1),  
    1000);  
  // X Забыли cleanup!  
}, []);
```

В dev: создаётся ДВА интервала!

Исправление:

```
useEffect(() => {  
  const id = setInterval(() => setCount(c => c + 1),  
    1000);  
  return () => clearInterval(id);  
}, []);
```

Хук `useRef` создает мутабельный объект с свойством `current`, который сохраняется между рендерами, но в отличие от состояния его изменение не вызывает перерендер. Это делает `ref` идеальным для хранения значений, которые не должны влиять на интерфейс - идентификаторов таймеров, предыдущих значений, флагов монтирования или экземпляров внешних объектов.

1. управление фокусом:

```
const inputRef = useRef(null);
useEffect(() => {
  inputRef.current?.focus();
}, []);
const handleClear = () => {
  inputRef.current.value = '';
  inputRef.current.focus();
};
```

2. прокрутка к элементу:

```
const sectionRef = useRef(null);
const scrollToSection = () => {
  sectionRef.current?.scrollIntoView({
    behavior: 'smooth',
    block: 'start'
  });
};
```

Ссылки и `useRef`

Основное применение `useRef` - получение прямого доступа к DOM-элементам. Передав `ref` в атрибут элемента, после монтирования в `current` окажется ссылка на настоящий DOM-узел, с которым можно работать императивно - устанавливать фокус, прокручивать, измерять размеры. Однако использовать `ref` для изменения содержимого элементов не рекомендуется - для этого существуют `useState` и `JSX`.

Когда вызывается:

1. При монтировании: `callbackRef(element)`
2. При размонтировании: `callbackRef(null)`
3. При изменении функции: `null → новая(element)`

Когда использовать:

- Действия в момент присвоения
- Динамические списки `refs`
- Уведомление о размонтировании

`useRef`:

```
const inputRef = useRef(null);
useEffect(() => {
  inputRef.current?.focus();
}, []);

```

Callback ref:

```
<input ref={(node) => node?.focus()} />
```

Ссылки и `useRef`

Callback ref представляет альтернативный подход, когда в качестве ref передается функция, вызываемая с элементом при монтировании и с null при размонтировании. Это дает возможность выполнять действия непосредственно в момент присоединения элемента и особенно полезно для динамических списков ссылок.

Когда вызывается:

1. При монтировании: `callbackRef(element)`
2. При размонтировании: `callbackRef(null)`
3. При изменении функции: `null → новая(element)`

Когда использовать:

- Действия в момент присвоения
- Динамические списки refs
- Уведомление о размонтировании

useRef:

```
const inputRef = useRef(null);
useEffect(() => {
  inputRef.current?.focus();
}, []);

```

Callback ref:

```
<input ref={(node) => node?.focus()} />
```

Ссылки и useRef

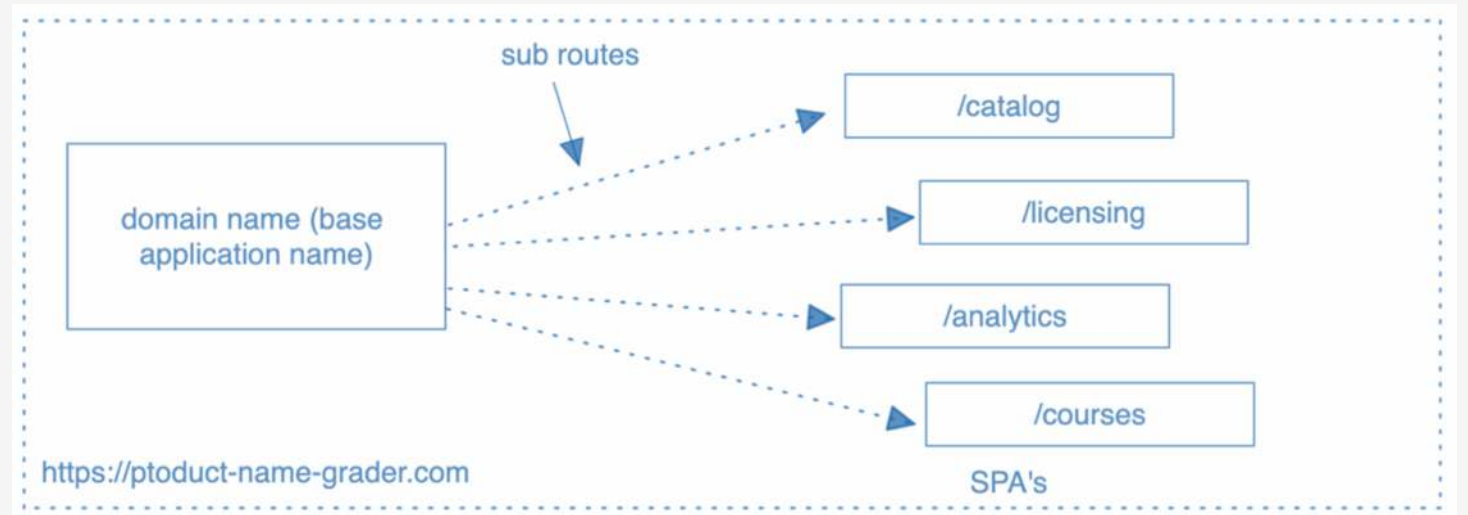
Подъем состояния и кастомные хуки

Когда несколько компонентов должны работать с одними данными, состояние поднимается до ближайшего общего предка. Этот предок хранит данные и передает их вниз через пропсы, а также передает дочерним компонентам функции для обновления состояния. Такой паттерн обеспечивает единый источник истины и синхронизацию всех компонентов, использующих общие данные. При этом данные всегда текут вниз по пропсам, а события - вверх через вызовы переданных функций.

Подъем состояния и кастомные хуки

Кастомные хуки позволяют выносить повторяющуюся логику из компонентов в переиспользуемые функции с префиксом `use`. Внутри кастомного хука можно использовать любые встроенные хуки React, а каждый вызов хука создает изолированное состояние. Это мощный механизм для абстрагирования работы с `localStorage`, дебаунса, форм или любой другой логики, которая нужна в нескольких компонентах. Кастомные хуки делают компоненты чище и код приложения более организованным.

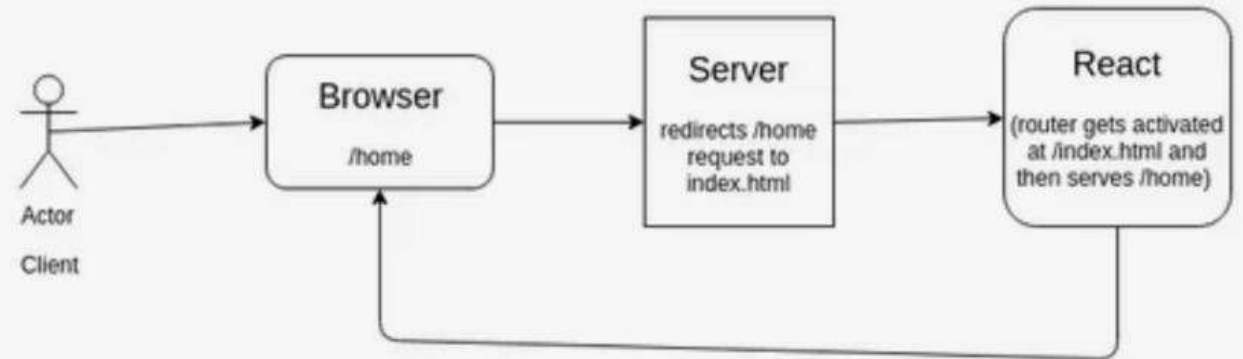
В классических многостраничных сайтах каждый переход по ссылке загружает новый HTML-документ с сервера, вызывая полную перезагрузку страницы. В одностраничных приложениях на React документ остается неизменным, а меняется только компонент, отображаемый в основной области. Маршрутизация решает задачу сопоставления URL в браузере с конкретным компонентом, который должен быть показан пользователю.



Маршрутизация в одностраничных приложениях

React

Это дает несколько важных преимуществ. Во-первых, появляется адресуемость экранов - каждый экран получает стабильный адрес, которым можно поделиться или сохранить в закладки. Во-вторых, переходы выполняются без перезагрузки, что создает ощущение плавного приложения. В-третьих, появляется возможность централизованно обрабатывать ошибки и проверки доступа - если у пользователя нет прав на страницу, его можно мягко перенаправить на другую.



Маршрутизация в одностраничных приложениях

React

Мини-словарь React Router

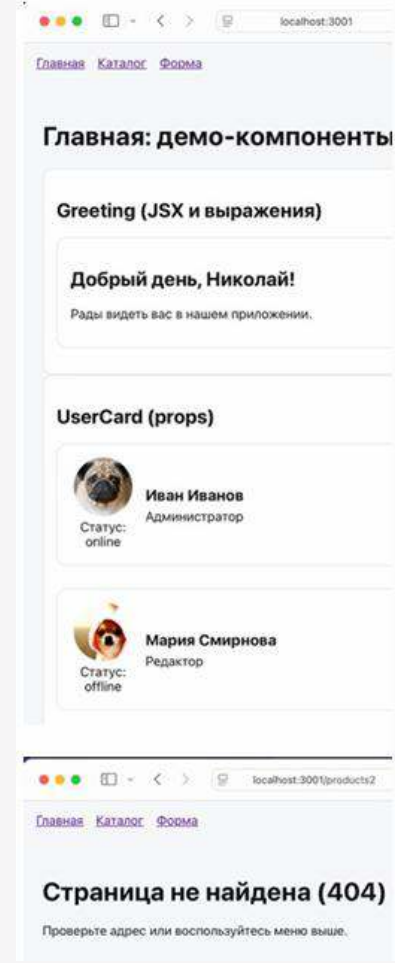
Термин	Что делает и где используется
BrowserRouter	Оборачивает приложение и «слушает» изменение URL. Без него роутинг не работает.
Routes	Контейнер правил сопоставления пути и компонента.
Route	Конкретное правило: если path совпал — отрисовать element.
Link	Переход внутри SPA без перезагрузки документа.
NavLink	Как Link, но с автоматической подсветкой активного пункта (aria-current="page").
useParams	Хук: читает динамические сегменты пути (:id, :slug).
Navigate	Программное перенаправление по условию.
ProtectedRoute	Обёртка: если доступ запрещён — вернуть <Navigate .../> (полная авторизация позже).

Маршрутизация в одностраничных приложениях

React

Библиотека React Router предоставляет набор компонентов для реализации маршрутизации. BrowserRouter оборачивает все приложение и обеспечивает синхронизацию интерфейса с адресной строкой. Routes и Route задают правила сопоставления путей и компонентов. Link и NavLink создают ссылки для навигации без перезагрузки, причем NavLink автоматически добавляет атрибуты для стилизации активного пункта меню.

```
9 export default function App() {
10   return (
11     <BrowserRouter>
12       <nav aria-label="Главное меню" style={{ display: "flex", gap: 16, padding: 16 }}>
13         <NavLink to="/" end>Главная</NavLink>
14         <NavLink to="/products">Каталог</NavLink>
15         <NavLink to="/form">Форма</NavLink>
16       </nav>
17
18       <Routes>
19         <Route path="/" element={<Home />} />
20         <Route path="/products" element={<ProductsList />} />
21         <Route path="/products/:id" element={<ProductDetails />} />
22         <Route path="/form" element={<FormsDemo />} />
23         <Route path="*" element={<NotFound />} />
24       </Routes>
25     </BrowserRouter>
26   );
27 }
```



Маршрутизация в одностраничных приложениях

Динамические маршруты и передача данных

В реальных приложениях часто требуются маршруты с переменными частями, например `/products/42` или `/users/ivan`.

Для этого в определении маршрута используется двоеточие перед именем параметра - `/products/:id`. В компоненте, который рендерится по такому маршруту, хук `useParams` возвращает объект со значениями всех параметров текущего URL. Важно помнить, что все значения из параметров приходят в виде строк, поэтому при необходимости работы с числами требуется явное преобразование.

```
src > App.js > App
1 // src/App.js — каркас под лекцию 6: Router + формы
2 import { BrowserRouter, Routes, Route, NavLink } from "react-router-dom";
3 import Home from "../pages/Home.jsx";
4 import ProductsList from "../pages/ProductsList.jsx";
5 import ProductDetails from "../pages/ProductDetails.jsx";
6 import FormsDemo from "../pages/FormsDemo.jsx";
7 import NotFound from "../pages/NotFound.jsx";
8
9 export default function App() {
10   return (
11     <BrowserRouter>
12       <nav aria-label="Главное меню" style={{ display: "flex", gap: 16, padding: 16 }}>
13         <NavLink to="/" end>Главная</NavLink>
14         <NavLink to="/products">Каталог</NavLink>
15         <NavLink to="/form">Форма</NavLink>
16       </nav>
17
18       <Routes>
19         <Route path="/" element={<Home />} />
20         <Route path="/products" element={<ProductsList />} />
21         <Route path="/products/:id" element={<ProductDetails />} />
22         <Route path="/form" element={<FormsDemo />} />
23         <Route path="*" element={<NotFound />} />
24       </Routes>
25     </BrowserRouter>
26   );
27 }
```

React

Динамические маршруты и передача данных

The image illustrates the implementation of dynamic routes and data passing in a React application. It features a code editor on the left and three browser window previews on the right.

Code Editor (ProductDetails.js):

```
src > pages > ProductDetails.jsx > ProductDetails
1 // src/pages/ProductDetails.jsx
2 import { useParams, useLocation, Link } from "react-router-dom";
3
4 export default function ProductDetails() {
5   const { id } = useParams();
6   const { state } = useLocation();
7   const previewName = state?.preview?.name;
8
9   return (
10     <article style={{ maxWidth: 960, margin: "0 auto", padding: 24 }}>
11       <h1>Товар #{id}</h1>
12       <p>
13         {previewName ? `Предпросмотр: ${previewName}` : "Данные подгрузим из API..."}
14       </p>
15       <p><Link to="/products">← Вернуться к каталогу</Link></p>
16     </article>
17   );
18 }
```

Browser Window 1 (localhost:3001/products/p1):

Главная Каталог Форма

Товар #p1

Данные подгрузим из API...

[← Вернуться к каталогу](#)

Главная Каталог Форма

Каталог

- [Ноутбук](#)
- [Монитор](#)
- [Клавиатура](#)

Browser Window 2 (localhost:3001/products/p2):

Главная Каталог Форма

Товар #p2

Предпросмотр: Монитор

[← Вернуться к каталогу](#)

Browser Window 3 (localhost:3001/products/p3):

Главная Каталог Форма

Товар #p3

Данные подгрузим из API...

[← Вернуться к каталогу](#)

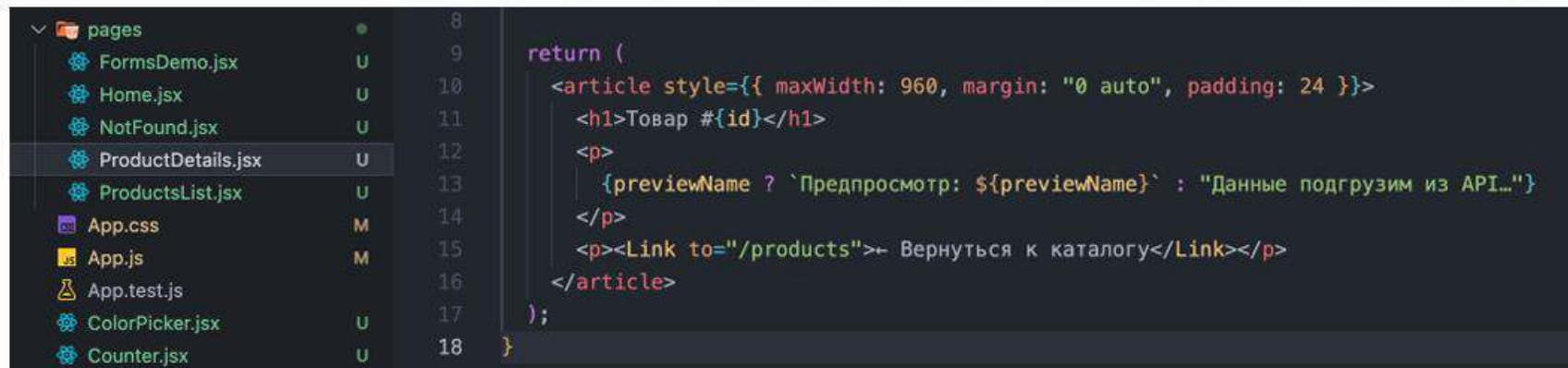
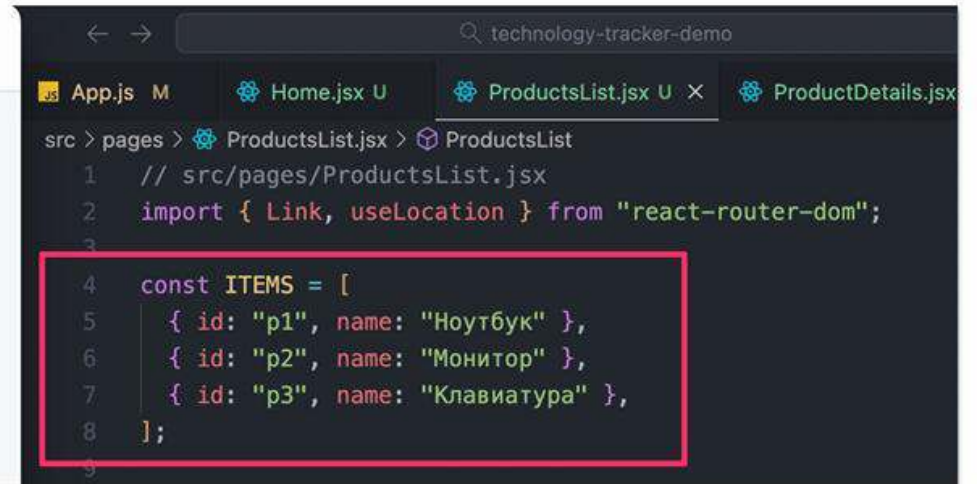
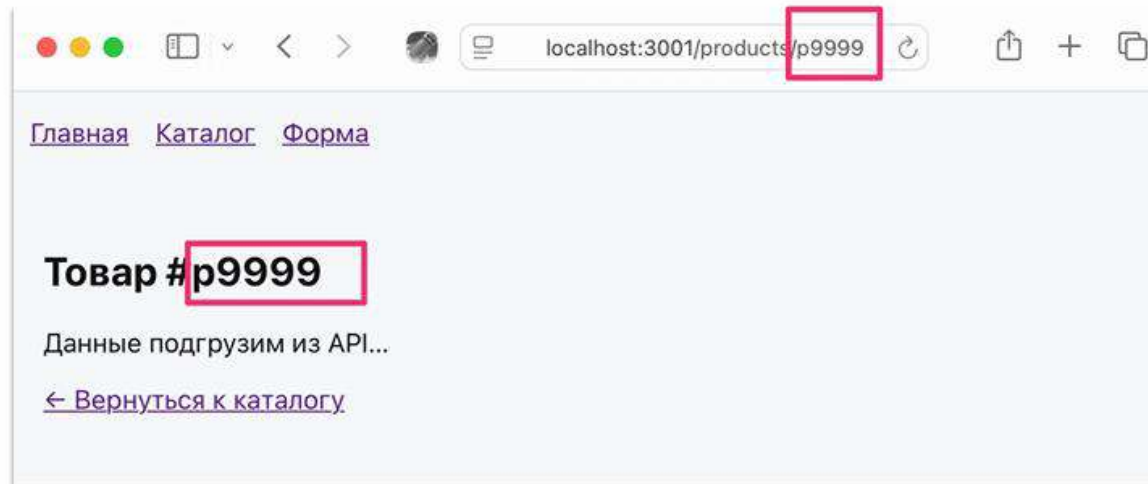
Динамические маршруты и передача данных

При навигации между страницами через `Link` можно передавать не только целевой путь, но и дополнительные данные через свойство `state`. Это удобно для мгновенного отображения предпросмотра при переходе со списка на детальную страницу - вместе с переходом передается объект с данными элемента, и страница деталей может сразу показать название и описание, параллельно запрашивая полные данные с сервера. При прямом вводе URL состояние отсутствует, и компонент сразу переходит к загрузке.

Важно реализовать корректное поведение при возврате на предыдущую страницу. Передавая вместе с переходом идентификатор элемента, можно после возврата прокрутить список к нужному месту и подсветить выбранный элемент, что создает целостный пользовательский опыт без потери контекста.

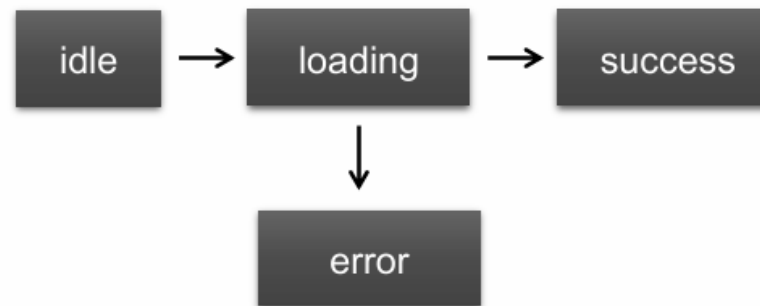
React

Динамические маршруты и передача данных



Любое асинхронное взаимодействие с данными проходит через несколько состояний - начальное состояние ожидания, загрузка, успешное получение данных или ошибка. Эти состояния должны быть единым контрактом между логикой и интерфейсом. Рекомендуется использовать три связанные сущности: статус, данные и ошибку. Статус принимает значения `idle`, `loading`, `success` или `error` и управляет тем, что отображается пользователю.

Мини state-machine для любого асинхронного процесса



Состояния загрузки и предсказуемый интерфейс

Для каждого статуса предусмотрено свое визуальное представление. При загрузке показывается скелетон — каркас будущего контента, который занимает примерно те же размеры и предотвращает нежелательные смещения верстки. При ошибке выводится понятное сообщение с возможностью повторить попытку. При успешной загрузке отображаются сполученные данные. Важно обеспечить доступность этих состояний для скринридеров - с помощью атрибутов `role="status"` для загрузки и `role="alert"` для ошибок.

```
src > entities > model >  useProduct.js >  useProduct
1  // src/entities/product/model/useProduct.js
2  import { useEffect, useState } from "react";
3  import { productRepo } from "../api/repo";
4
5  export function useProduct(id, preview) {
6    const [data, setData] = useState(preview || null);
7    const [status, setStatus] = useState(preview ? "success" : "loading");
8    const [error, setError] = useState(null); // + error
9  }
```

Состояния загрузки и предсказуемый интерфейс

Для работы с асинхронными запросами рекомендуется использовать `AbortController`, чтобы отменять запросы при размонтировании компонента или изменении параметров. Это предотвращает проблемы с утечками памяти и обновлением состояния размонтированного компонента.

Состояния загрузки и предсказуемый интерфейс

React

Формы и валидация

React предлагает два основных подхода к работе с формами. Первый - использование `FormData`, при котором данные собираются непосредственно из формы в момент отправки без хранения в состоянии. Это простой и эффективный способ для форм, где не требуется мгновенная валидация или сложное взаимодействие между полями. Второй подход - контролируемые компоненты, где каждое поле связано с состоянием React через `value` и `onChange`, что дает полный контроль над вводом и возможность реализовать любую логику валидации.

Формы и валидация

Для более сложных форм лучше использовать библиотеку React Hook Form, которая устраняет необходимость ручного управления состоянием каждого поля. Поля регистрируются через функцию `register`, ошибки валидации автоматически собираются в объекте `errors`, а схему валидации удобно описывать с помощью библиотеки `Zod`. Такой подход дает компактный, декларативный код со встроенной поддержкой доступности.

Критерий	Controlled	React Hook Form
Управление состоянием	Через <code>useState</code> для каждого поля	Автоматически через внутренний стор
Производительность	Каждый ввод вызывает <code>re-render</code>	Оптимизирован: поля подписаны на свои данные
Валидация	Ручная (<code>if</code> , <code>useEffect</code>)	Через <code>resolver</code> (например, <code>Zod</code>)
Простота	Подходит для 1–2 полей	Подходит для сложных форм
Поддержка <code>a11y</code>	Требует ручной разметки ролей	Встроенная поддержка <code>aria</code> -атрибутов

Формы и валидация

При реализации форм важно уделять внимание доступности. Поля с ошибками должны получать атрибут `aria-invalid`, сообщения об ошибках должны быть связаны с полями через `aria-describedby` и иметь `role="alert"`, чтобы скринридеры немедленно объявляли о проблемах. Теги `fieldset` и `legend` помогают группировать связанные элементы формы, например радиокнопки или чекбоксы.

Для изоляции работы с внешними данными рекомендуется использовать паттерн репозиторий. Все API-вызовы выносятся в отдельные функции, которые компоненты вызывают через пользовательские хуки. Это позволяет легко заменять реальное API на фейковое для тестирования или разработки, не меняя код компонентов. Компоненты работают только с данными и статусами загрузки, не зная деталей их получения.

Работа с API и архитектурные паттерны

React

Такой подход хорошо сочетается с архитектурой, напоминающей Feature-Sliced Design. Проект организуется по слоям: в `shared` находятся переиспользуемые UI-компоненты и утилиты, в `entities` - модели данных и логика работы с ними, в `features` - готовые функциональные блоки, в `pages` - экранные компоненты, собирающие страницы из фич и сущностей, а в `app` - инициализация приложения и маршруты. Такая структура делает код предсказуемым и удобным для масштабирования.

Работа с API и архитектурные паттерны

React

Пользовательские хуки идеально подходят для инкапсуляции логики работы с данными. Хук `useProduct` принимает идентификатор и опциональные данные предпросмотра, управляет состояниями загрузки и ошибок, выполняет запросы с отменой при необходимости и возвращает готовый к использованию объект с данными и статусом. Компонент страницы остается тонким - он только читает параметры из URL и переданное состояние, вызывает хук и рендерит соответствующий интерфейс в зависимости от статуса.

Работа с API и архитектурные паттерны

React

4

VUE

Место Vue 3 в современной фронтенд-разработке

До этой лекции мы работали с чистым JavaScript, где каждое изменение интерфейса требовало ручных операций с DOM, а затем познакомились с React и его компонентным подходом на основе JSX и хуков. Vue 3 предлагает альтернативный маршрут к решению тех же задач - построению сложных пользовательских интерфейсов, - но с другим философским подходом. Если React предлагает свободу и конструирование стека под проект, то Vue дает цельное, согласованное решение прямо из коробки.



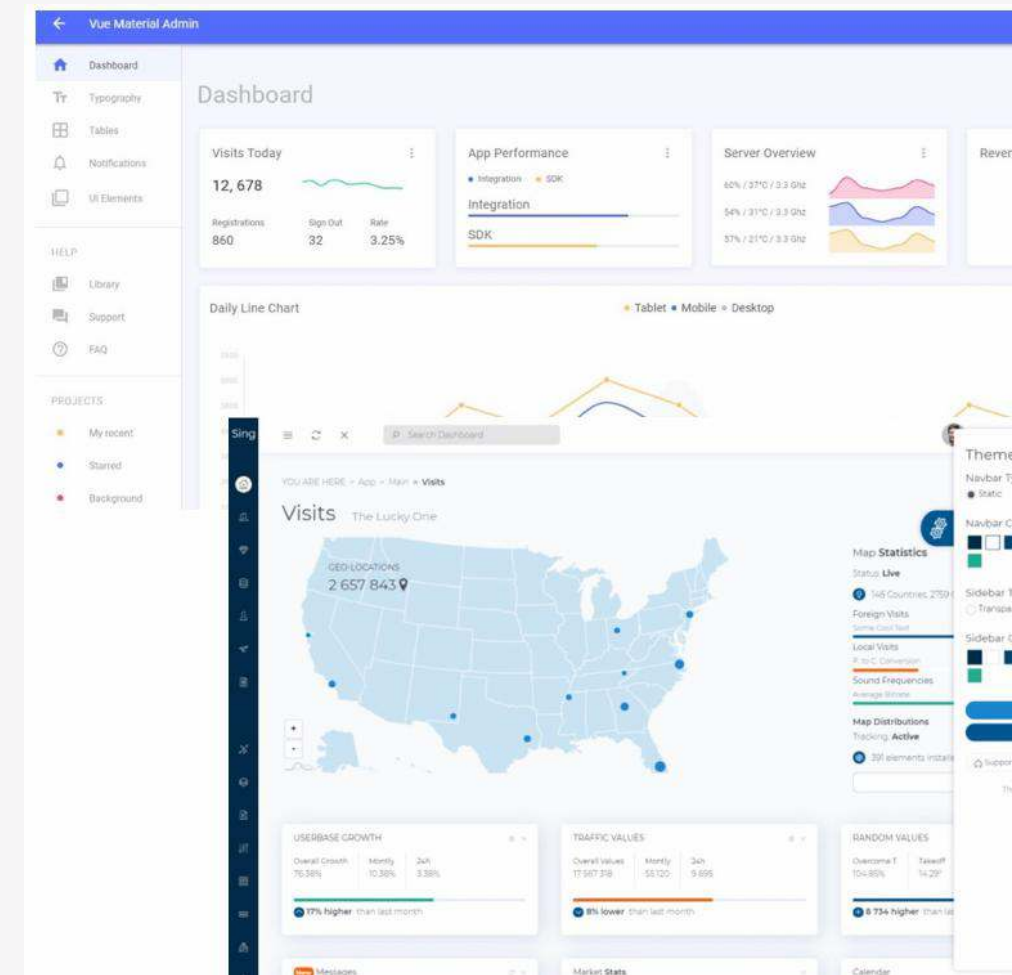
Место Vue 3 в современной фронтенд-разработке

Исторически Vue появился как ответ на перегруженность существующих фреймворков и желание соединить простоту jQuery с мощностью одностраничных приложений. Его создатели хотели получить более мягкий вход, чем в тяжеловесном Angular, и более предсказуемую архитектуру, чем в React с его принципом «собери всё сам». В результате Vue предлагает разработчику шаблоны, реактивность и компоненты уже в готовом, продуманном виде, что особенно ценно для команд, где важны читаемость кода и предсказуемость результатов.



На практике Vue 3 чаще всего выбирают для создания одностраничных приложений и личных кабинетов, где важна плавность взаимодействия и четкая структура кода.

Особенно хорошо фреймворк раскрывается во внутренних сервисах компаний – админках, дашбордах, CRM-системах и учетных приложениях, где интерфейс строится вокруг множества форм, фильтров, таблиц и интерактивных виджетов.



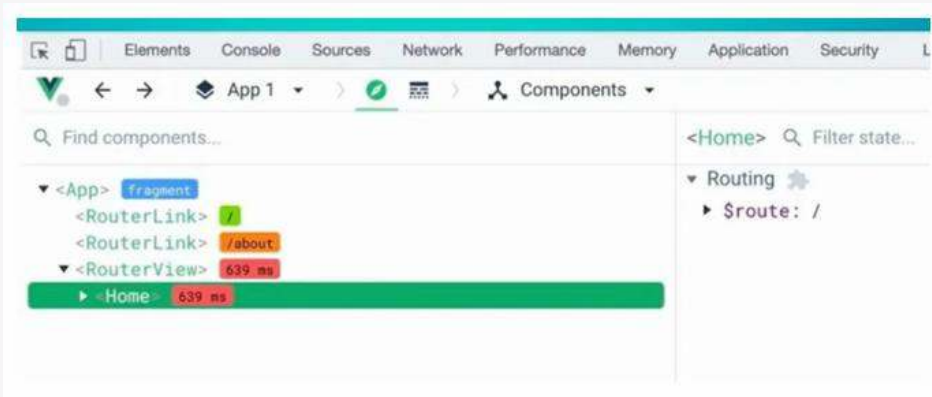
Где Vue 3 показывает себя лучше всего

Vue

Vue также отлично подходит для продуктов, где критически важны читаемость кода и скорость разработки - например, при создании интерактивных configurаторов или конструкторов интерфейсов. Типичный технологический стек для таких проектов выглядит как Vite для сборки, Vue 3 как основа, Vue Router для навигации и Pinia для управления состоянием - и всё это официальные, хорошо сочетающиеся друг с другом инструменты.



Pinia



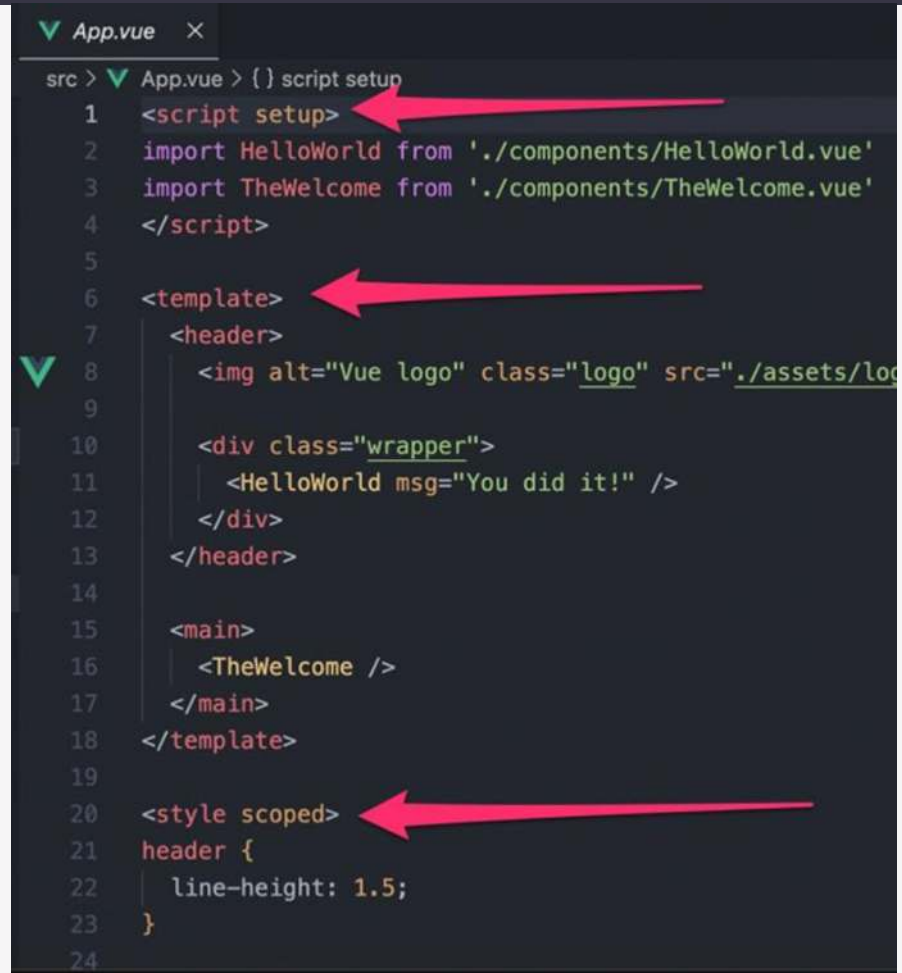
Где Vue 3 показывает себя лучше всего

Vue

Три фундаментальных принципа Vue

В основе Vue лежат три взаимосвязанных понятия, которые вместе образуют архитектуру фреймворка. Первое – это компоненты, логически цельные блоки интерфейса, которые работают как переиспользуемые строительные кирпичики приложения.

Второе - шаблоны, которые дают возможность декларативно описывать интерфейс и напрямую связывать его с данными из логики компонента.



```
App.vue x
src > App.vue > {} script setup
1 <script setup>
2 import HelloWorld from './components/HelloWorld.vue'
3 import TheWelcome from './components/TheWelcome.vue'
4 </script>
5
6 <template>
7   <header>
8     
9
10    <div class="wrapper">
11      <HelloWorld msg="You did it!" />
12    </div>
13  </header>
14
15  <main>
16    <TheWelcome />
17  </main>
18 </template>
19
20 <style scoped>
21   header {
22     line-height: 1.5;
23   }
24
```

Три фундаментальных принципа Vue

И третье, самое важное – реактивность. Vue автоматически отслеживает все данные, которые разработчик пометил как реактивные, и подписывает на них соответствующие части шаблона. Когда данные меняются, фреймворк самостоятельно определяет, какие именно элементы интерфейса нужно обновить, и делает это без участия разработчика. В результате мы перестаем думать о том, как обновлять DOM, и сосредотачиваемся только на том, как должны меняться данные в ответ на действия пользователя.

React и Vue 3 преследуют одни и те же цели — создание интерфейсов через компонентный подход и декларативное описание, но идут к ним разными путями. React позиционирует себя как библиотеку с минимальным ядром, предлагая разработчику самостоятельно собирать остальной стек под конкретный проект, используя функциональные компоненты и JSX для описания интерфейсов.



Vue 3, напротив, является полноценным фреймворком с более цельным подходом. Вместо JSX здесь используются шаблоны и формат Single File Components, где разметка, логика и стили живут в одном файле. Для работы с состоянием React предлагает хуки `useState`, `useEffect` и другие, тогда как в Vue эту роль выполняют `ref` и `reactive` для хранения данных, `computed` для вычисляемых значений и `watch` для реакций на изменения. При этом у Vue есть официальные инструменты для большинства типовых задач - Vue Router и Pinia, - тогда как в React экосистема строится из множества независимых решений.

Vue и React – общие цели, разная реализация

Vue

Входной порог и опыт разработчика

Для новой команды или проекта выбор между React и Vue часто определяется не столько техническими возможностями, сколько порогом входа и предсказуемостью разработки. В React, чтобы начать, нужно не только уверенно владеть современным JavaScript, но и принять философию JSX с мышлением в терминах функций, а также сделать непростой выбор стартового стека из множества вариантов - Vite, Create React App, Next.js и других.

Vue 3 предлагает значительно более мягкий старт благодаря цельному подходу. Разработчик получает готовый официальный стек из Vite, самого Vue, Router и Pinia, подкрепленный отличными инструментами разработчика вроде Vue DevTools. Формат Single File Components со специальным режимом script setup делает код интуитивно понятным – все, что объявлено в скрипте, сразу доступно в шаблоне. В результате команда тратит меньше времени на архитектурные дискуссии и быстрее переходит к написанию рабочего кода.

Для работы с Vue 3 необходим Node.js — среда, позволяющая запускать JavaScript вне браузера и управлять пакетами через менеджеры npm или yarn. Vite, который мы используем для сборки проекта, работает именно поверх Node.js, поэтому перед началом стоит убедиться, что установлена актуальная LTS-версия - проверить это можно командами `node -v` и `npm -v` в терминале.

```
> npx
> create-vue

Vue.js - The Progressive JavaScript Framework

◇ Project name (target directory):
  vue-mini-dashboard

◇ Select features to include in your project: (↑/↓ to navigate, space to select, a to toggle all, enter to confirm)
  none

◇ Select experimental features to include in your project: (↑/↓ to navigate, space to select, a to toggle all, enter to confirm)
  none

◇ Skip all example code and start with a blank Vue project?
  No
```

Начало работы – Node.js и создание проекта

Vue

Создание нового проекта на Vue 3 с Vite выполняется одной командой `npm create vue@latest`, после чего нужно ответить на несколько вопросов о имени проекта и желаемых опциях вроде поддержки TypeScript. На выходе мы получаем готовую структуру папок с `package.json` для зависимостей, конфигурацией Vite и папкой `src`, где будет находиться весь исходный код. Остается только перейти в папку проекта, установить зависимости через `npm install` и запустить сервер разработки командой `npm run dev`.

```
> vue-mini-dashboard@0.0.0 dev
> vite

VITE v7.2.4 ready in 479 ms

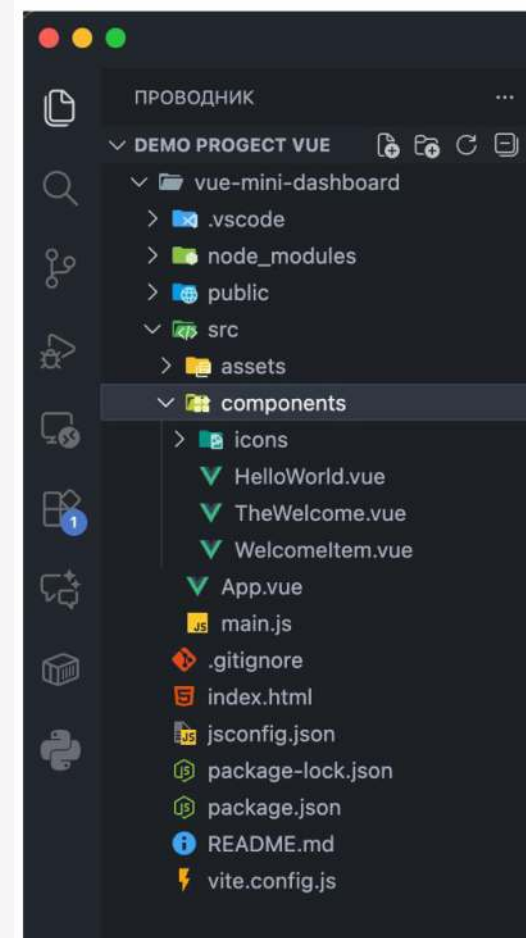
→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ Vue DevTools: Open http://localhost:5173/__devtools__/ as a separate window
→ Vue DevTools: Press Option(⌘)+Shift(⇧)+D in App to toggle the Vue DevTools
→ press h + enter to show help
```

Начало работы – Node.js и создание проекта

Vue

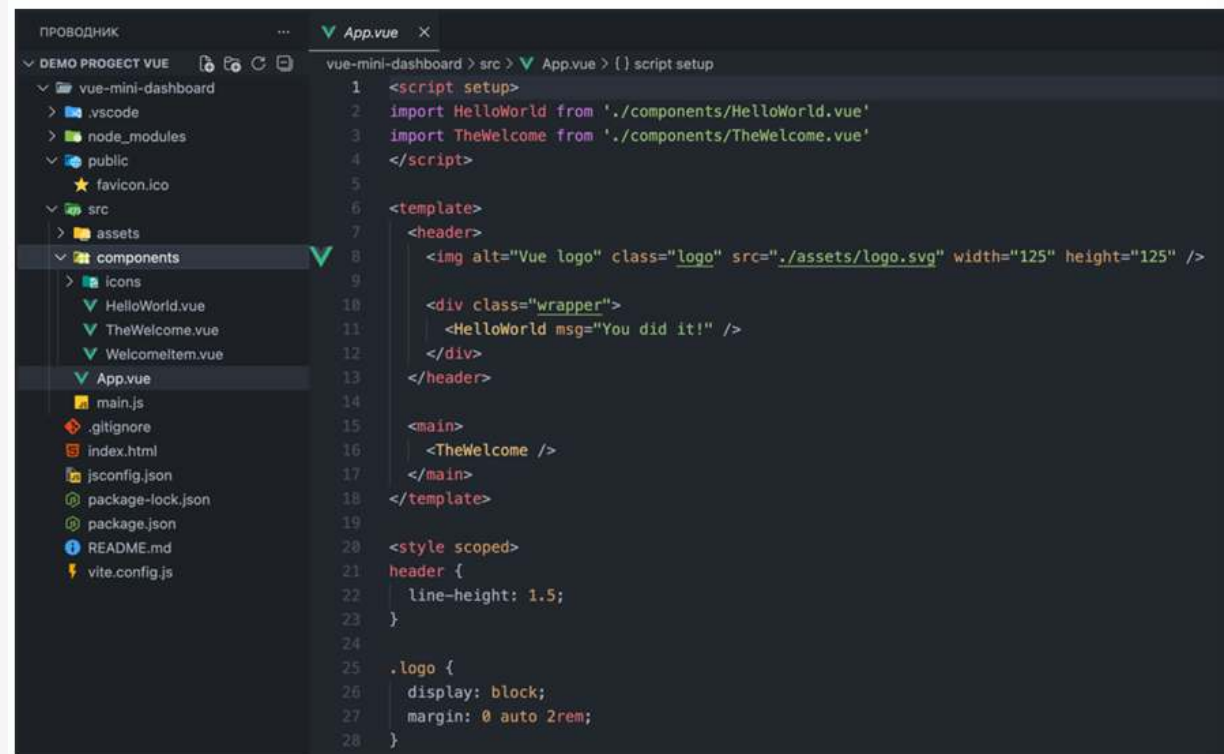
Single File Components. Компонент во Vue

Главная особенность Vue, которую вы будете видеть каждый день — это формат Single File Component, или SFC. Каждый компонент представляет собой файл с расширением `.vue`, внутри которого в четкой структуре объединены три составляющие. Блок `template` содержит HTML-разметку с особыми директивами Vue для привязки данных. Блок `script setup` описывает логику и состояние компонента на чистом JavaScript — причем все объявленные здесь переменные и функции автоматически становятся доступны в шаблоне. Блок `style` отвечает за стили, которые при необходимости можно изолировать только для этого компонента, добавив атрибут `scoped`.



Single File Components. Компонент во Vue

Важно понимать, что браузер не умеет работать с файлами .vue напрямую — на этапе сборки Vite преобразует их в понятный для браузера JavaScript и CSS. Шаблон превращается в функцию рендера, скрипт становится модулем с логикой, а стили собираются в общий CSS-бандл. Но для разработчика вся работа происходит именно с удобными .vue-файлами, где не нужно искать HTML, JavaScript и CSS по разным папкам проекта.



Vue

App.vue

vue-mini-dashboard > src > App.vue > {} script setup

```
1 <script setup>
2 import HelloWorld from './components/HelloWorld.vue'
3 import TheWelcome from './components/TheWelcome.vue'
4 </script>
```

Логика

```
6 <template>
7   <header>
8     
9
10    <div class="wrapper">
11      <HelloWorld msg="You did it!" />
12    </div>
13  </header>
14
15  <main>
16    <TheWelcome />
17  </main>
18 </template>
```

Разметка

```
20 <style scoped>
21 header {
22   line-height: 1.5;
23 }
24
25 .logo {
26   display: block;
27   margin: 0 auto 2rem;
28 }
```

Стили

Сердце любого Vue-приложения - реактивные данные, и для их создания фреймворк предлагает два основных инструмента. Функция `ref` создает контейнер для одного реактивного значения - чаще всего это примитивы вроде чисел, строк или булевых значений. В коде JavaScript обращение к такому значению идет через свойство `value`, например `count.value`, но в шаблоне Vue автоматически разворачивает `ref`, и мы пишем просто `{{ count }}`.

```
CounterExample.vue x App.vue 7
vue-mini-dashboard > src > components > CounterExample.vue > {} script setup
1 <template>
2   <section class="panel">
3     <h2>Пример счётчика</h2>
4     <p>Текущее значение: {{ count }}</p>
5     <button type="button" @click="increment">
6       Увеличить
7     </button>
8   </section>
9 </template>
10
11 <script setup>
12   // Подключаем инструменты реактивности из Vue
13   import { ref } from "vue"
14
15   // Реактивная переменная счётчика
16   const count = ref(0)
17
18   // Обработчик клика по кнопке
19   function increment() {
20     count.value++ // меняем значение → Vue обновит разметку
21   }
22 </script>
23
```

Реактивность через `ref` и `reactive`

Для более сложных структур — объектов или массивов — удобнее использовать `reactive`. Эта функция оборачивает исходный объект в `Proxy`, который отслеживает все обращения к свойствам и их изменения. В отличие от `ref`, здесь не нужно использовать `.value` — мы работаем напрямую с полями объекта: `state.count`, `state.tasks`. Но важно помнить ограничение: нельзя переопределить весь реактивный объект целиком или деконструировать его — это сломает реактивность. `Reactive` идеально подходит для хранения связанного набора данных внутри одного компонента, например счетчика и шага изменения или списка задач с параметрами фильтрации.

```
1 <template>
2   <section class="panel">
3     <h2>Пример счётчика на ref</h2>
4     <p>Текущее значение: {{ count }}</p>
5     <button type="button" @click="increment">
6       Увеличить
7     </button>
8   </section>
9 </template>
10
11 <script setup>
12   import { ref } from "vue"
13
14   const count = ref(0)
15
16   function increment() {
17     count.value++
18   }
19 </script>
20
21 <style scoped>
22   .panel {
23     padding: 1rem;
24   }
25 </style>
```

```
1 <script setup>
2   import { reactive } from "vue"
3
4   // Реактивный объект состояния
5   const state = reactive({
6     count: 0,
7     step: 1,
8   })
9
10  // Функция изменения состояния
11  function inc() {
12    state.count += state.step
13  }
14
15 </script>
16
17 <template>
18   <section class="panel">
19     <h2>Proxy и reactive</h2>
20     <p>Текущее значение: {{ state.count }}</p>
21     <button type="button" @click="inc">
22       Увеличить на {{ state.step }}
23     </button>
24   </section>
25 </template>
```

Реактивность через `ref` и `reactive`

Computed и watch

Производные данные и побочные эффекты

На основе реактивных данных часто нужно строить производные значения или реагировать на изменения. Для первого случая существует `computed` - функция, которая принимает вычисление и возвращает реактивное значение, автоматически пересчитываемое только когда меняются его зависимости. Это удобно для форматирования текста, подсчета статистики вроде количества выполненных задач или подготовки отфильтрованных списков. Важное свойство `computed` - кэширование результата до тех пор, пока входные данные не изменились.

```
vue-mini-dashboard > src > components > TaskStats.vue > {} script setup > totalCount
2   <script setup>
13  // Вычисляем общее количество задач
14  const totalCount = computed(() => taskState.tasks.length)
15
16  // Считаем, сколько задач выполнено
17  const doneCount = computed(() =>
18    taskState.tasks.filter(task => task.done).length
19  )
20
21  // Текстовый статус для интерфейса
22  const statusText = computed(() =>
23    `${doneCount.value} из ${totalCount.value} задач выполнено`
24  )
```

Computed и watch

Производные данные и побочные эффекты

Watch решает другую задачу - позволяет выполнять побочные эффекты в ответ на изменения реактивных данных. Типичные сценарии использования включают автоматическое сохранение состояния в localStorage или на сервер при изменении данных, отправку аналитических событий или логирование действий пользователя.

Watch может следить как за отдельным значением, так и за массивом значений или даже за определенным полем в реактивном объекте.

```
37
38 // watch: сохраняем задачи в localStorage при изменении
39 watch(
40   () => taskState.tasks,           // источник – массив задач
41   (newTasks) => {
42     localStorage.setItem(
43       "tasks",
44       JSON.stringify(newTasks)     // побочный эффект: запись в хранилище
45     )
46   },
47   { deep: true }                  // следим за вложенными изменениями
48 )
```

Vue предоставляет набор специальных атрибутов - директив, - которые связывают разметку с реактивными данными и делают шаблоны по-настоящему динамическими. Директива `v-bind`, сокращенно записываемая как двоеточие, привязывает реактивное значение к любому HTML-атрибуту - будь то `href` ссылки, класс элемента или его стили. Директива `v-on`, или `@`, вешает на элемент обработчики событий, вызывая функции из `script setup` при кликах, отправке форм и других действиях пользователя.

- `v-bind (:)` — привязка атрибутов и классов
- `v-on (@)` — обработка событий
- `v-if / v-else / v-show` — условный рендеринг
- `v-for` — цикл по массиву
- `v-model` — двусторонняя привязка форм

Директивы — связь шаблона и данных

Для условного отображения элементов используются `v-if` и `v-show` - первая полностью удаляет или создает элемент в DOM, вторая просто прячет его через CSS, оставляя в разметке. Для отрисовки списков предназначена директива `v-for`, которая обязательно требует указания уникального ключа через `:key` для эффективного обновления. И наконец, `v-model` предоставляет удобный синтаксис для двусторонней привязки данных в формах, автоматически связывая значение поля ввода с реактивной переменной и обновляя её при вводе пользователя.

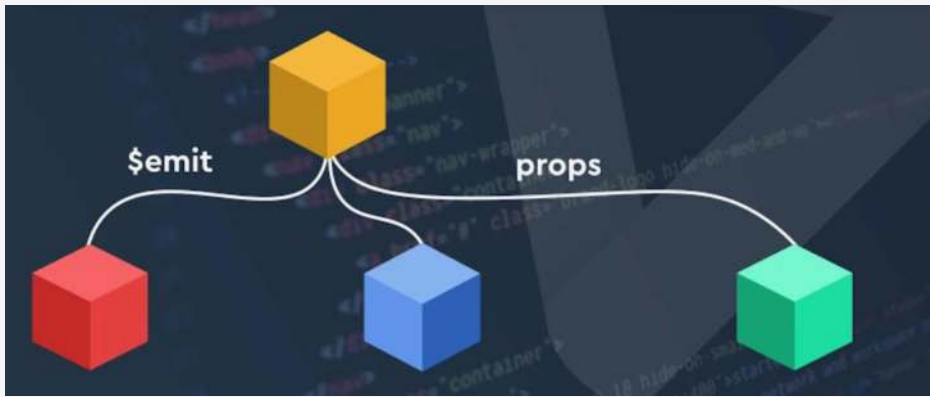
```
<label>
  <input type="checkbox" v-model="task.done" />
  {{ task.title }}
</label>
```

```
<li
  v-for="task in taskState.tasks"
  :key="task.id"
  :class="{ 'task-done': task.done }"
>
```

Директивы — связь шаблона и данных

Коммуникация между компонентами props, emits и слоты

Реальные приложения строятся из множества взаимодействующих компонентов, и Vue предлагает четкие механизмы для этой коммуникации. Родительский компонент передает данные дочерним через props, которые объявляются в дочернем компоненте с помощью defineProps. Props реактивны, но доступны только для чтения - дочерний компонент никогда не должен изменять полученные данные напрямую, это нарушило бы принцип однонаправленного потока.



```
<template>
  <li :class="{ 'task-list__item--done': props.task.done }">
    <label>
      <input
        type="checkbox"
        :checked="props.task.done"
        @change="onToggle"
      />
      {{ props.task.title }}
    </label>
    <button @click="onRemove">Удалить</button>
  </li>
</template>
```

Коммуникация между компонентами props, emits и слоты

Для отправки сообщений от ребенка к родителю служат emits - объявленные через defineEmits события, которые дочерний компонент инициирует функцией emit. Родитель подписывается на эти события через директиву @ и выполняет нужную логику, обычно изменяя данные, которые затем снова спускаются вниз через props. А когда нужно передать не данные, а целую разметку, на помощь приходят слоты - места в дочернем компоненте, куда родитель может вставить свой HTML, работающие по тому же принципу, что и children в React. Для более сложных случаев существуют именованные слоты, позволяющие заполнять несколько разных «дырок» в одном компоненте.

```
<script setup>

const props = defineProps({ task: Object })

const emit = defineEmits(['toggle', 'remove'])

function onToggle() {
  emit('toggle', props.task.id)
}

function onRemove() {
  emit('remove', props.task.id)
}

</script>
```

СПАСИБО ЗА ВНИМАНИЕ!

26 февраля 2026 года